CRYPTO 2 - Cours 3
Signatures de Schnorr
Partage de secret et calcul réparti sécurisé

Damien Vergnaud

Sorbonne Université – CNRS

Outline

1. Fiat-Shamir heuristic and Schnorr Signatures
   - Schnorr's Protocol
   - The Fiat-Shamir heuristic
   - Schnorr signatures
2. Secret-Sharing
   - Motivation
   - In the "Real-World"
   - Shamir's Secret Sharing
3. Multi-Party Computation
   - Motivation
   - Example: Mean computation
   - BGW Protocol – Semi-Honest Security

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.

P

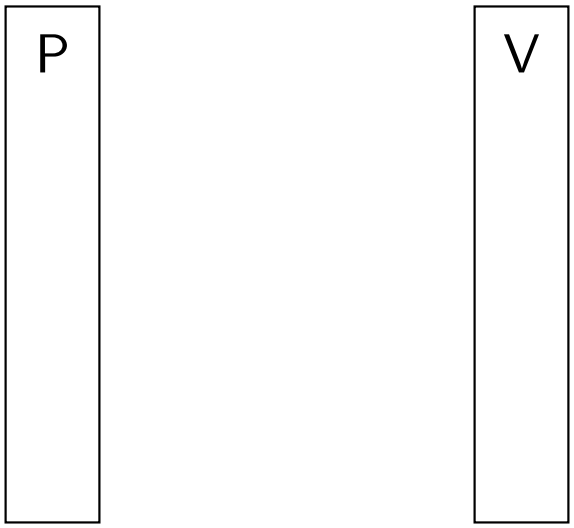
Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
V sends $c \xleftarrow{\$} \mathbb{Z}_q$
P sends $s = k + cx \bmod q$
V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.



Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
V sends $c \xleftarrow{\$} \mathbb{Z}_q$
P sends $s = k + cx \bmod q$
V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.

$x \xleftarrow{\$} \mathbb{Z}_q$

P

V

Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
V sends $c \xleftarrow{\$} \mathbb{Z}_q$
P sends $s = k + cx \bmod q$
V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.

$x \xleftarrow{\$} \mathbb{Z}_q$

$y = g^x$

P

V

Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$

V sends $c \xleftarrow{\$} \mathbb{Z}_q$

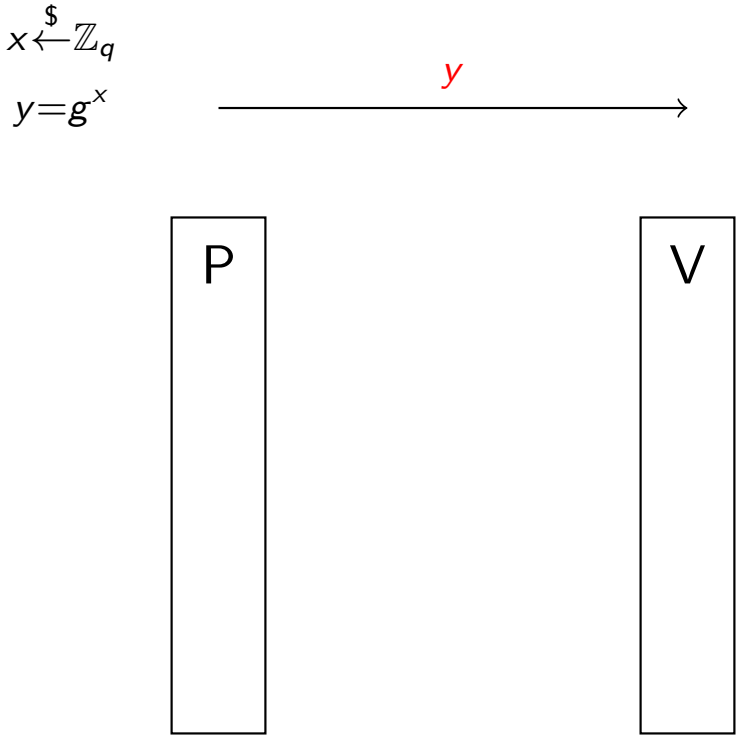
P sends $s = k + cx \bmod q$

V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.



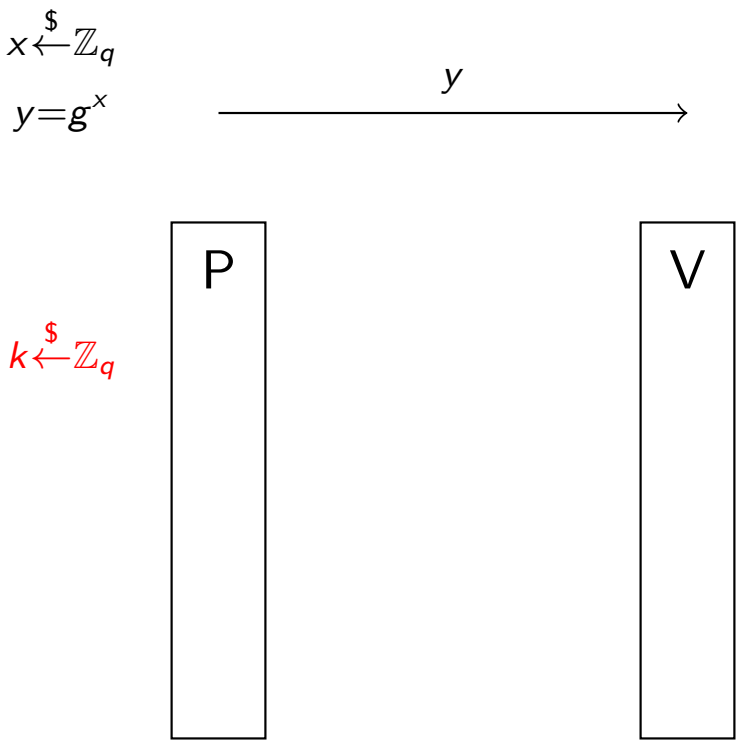
Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
V sends $c \xleftarrow{\$} \mathbb{Z}_q$
P sends $s = k + cx \bmod q$
V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.



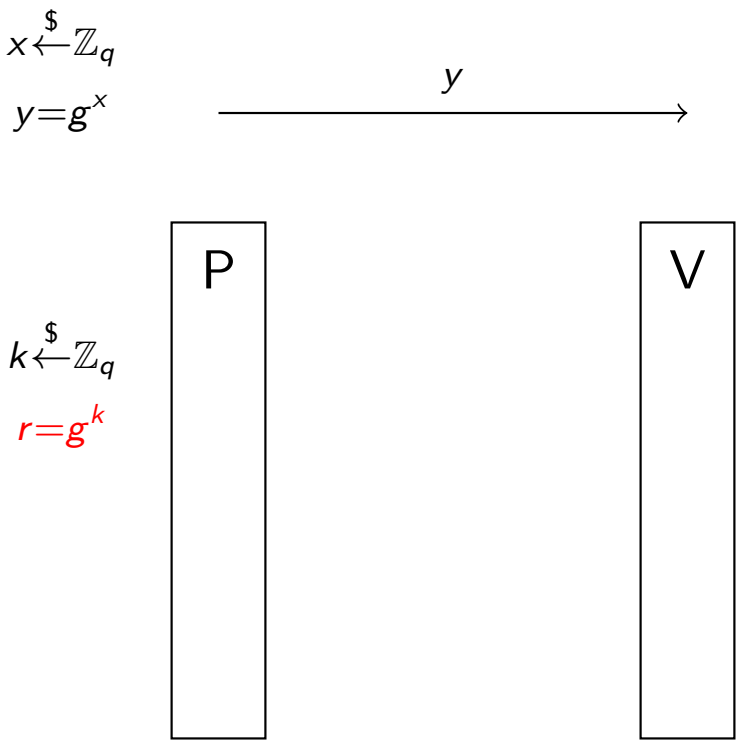
Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
V sends $c \xleftarrow{\$} \mathbb{Z}_q$
P sends $s = k + cx \bmod q$
V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.



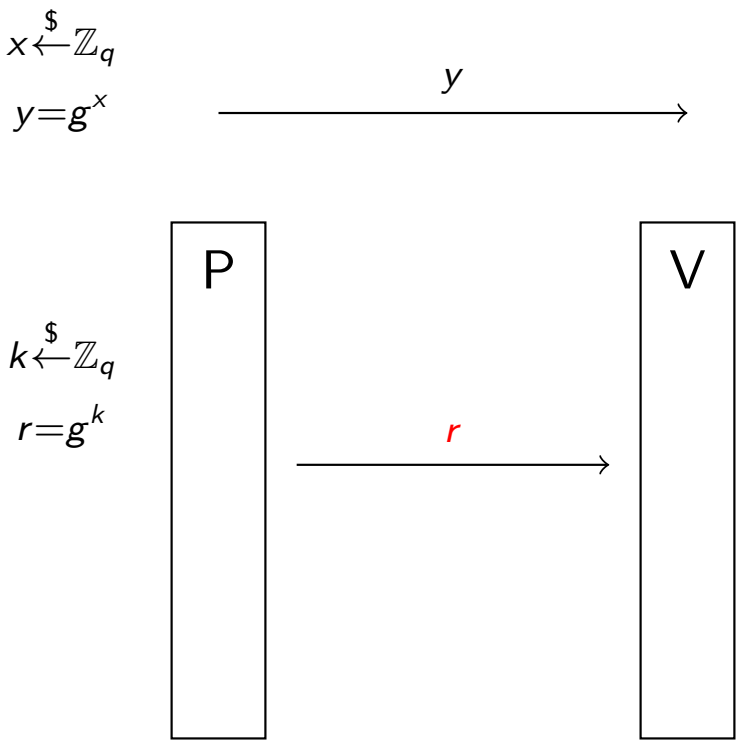
Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
V sends $c \xleftarrow{\$} \mathbb{Z}_q$
P sends $s = k + cx \bmod q$
V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.



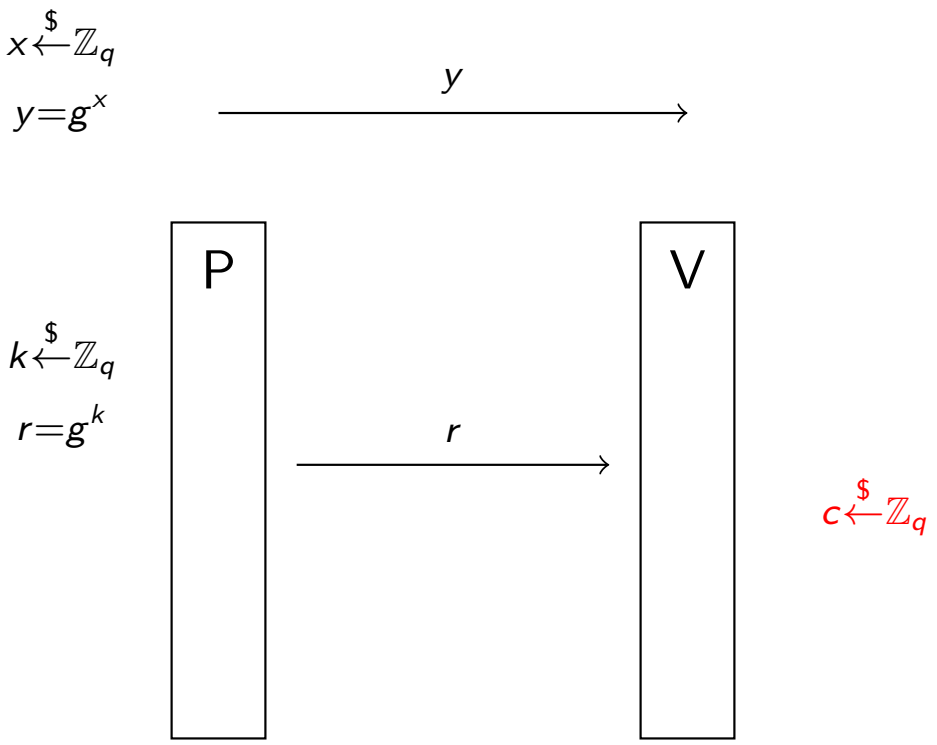
Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
V sends $c \xleftarrow{\$} \mathbb{Z}_q$
P sends $s = k + cx \bmod q$
V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.



Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$

V sends $c \xleftarrow{\$} \mathbb{Z}_q$

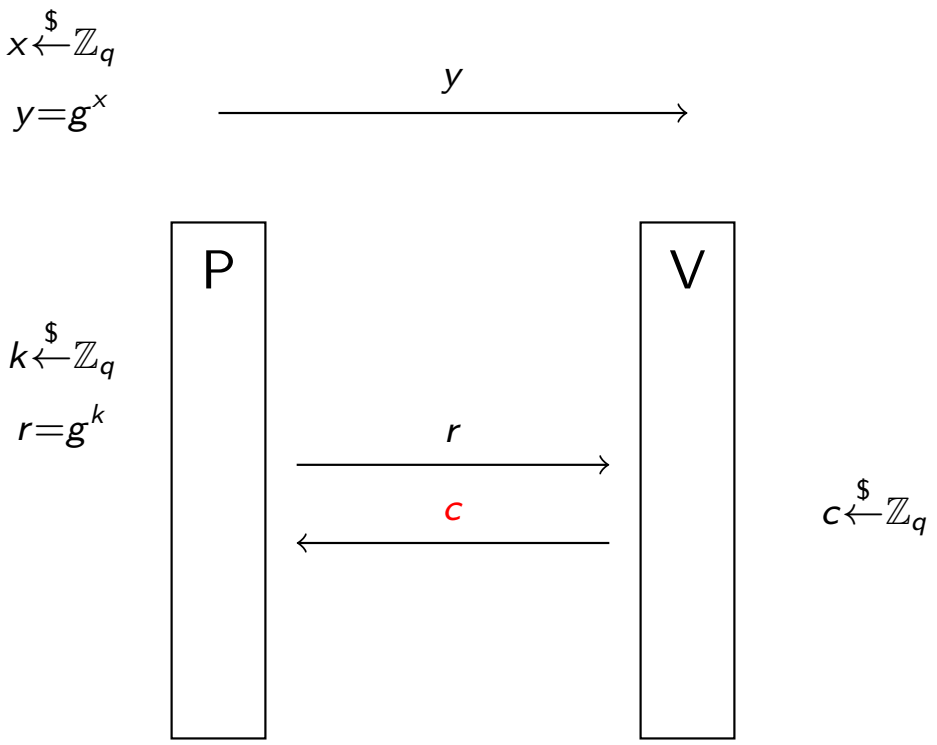
P sends $s = k + cx \bmod q$

V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.



Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$

V sends $c \xleftarrow{\$} \mathbb{Z}_q$

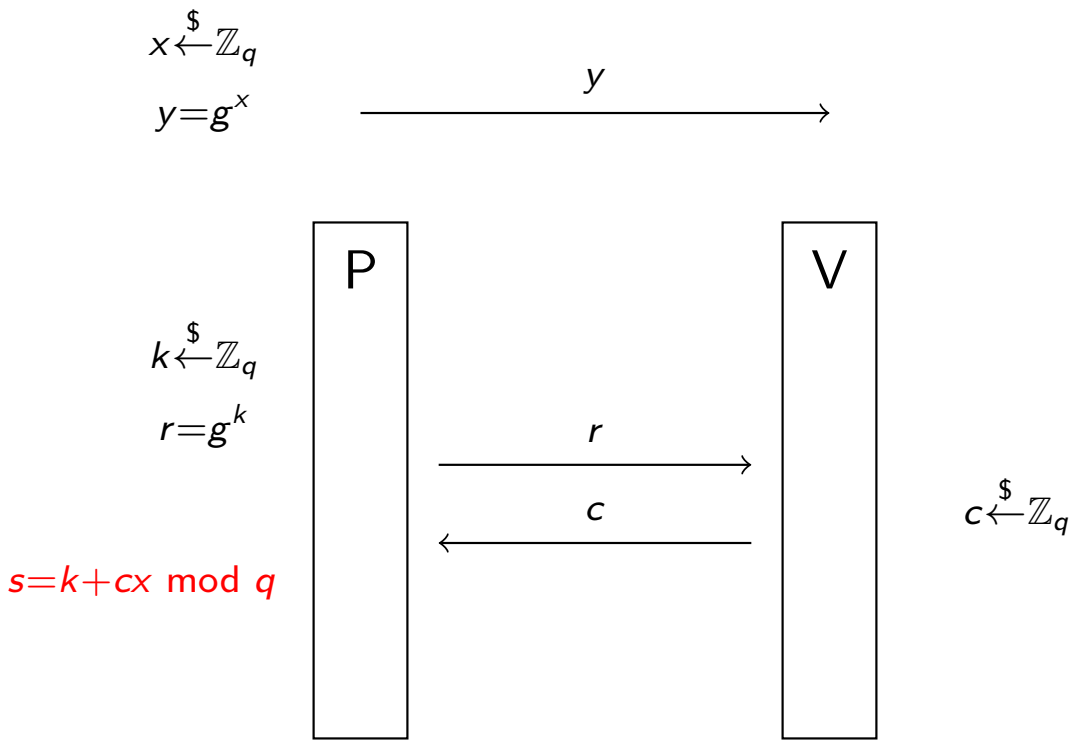
P sends $s = k + cx \bmod q$

V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.



Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$

V sends $c \xleftarrow{\$} \mathbb{Z}_q$

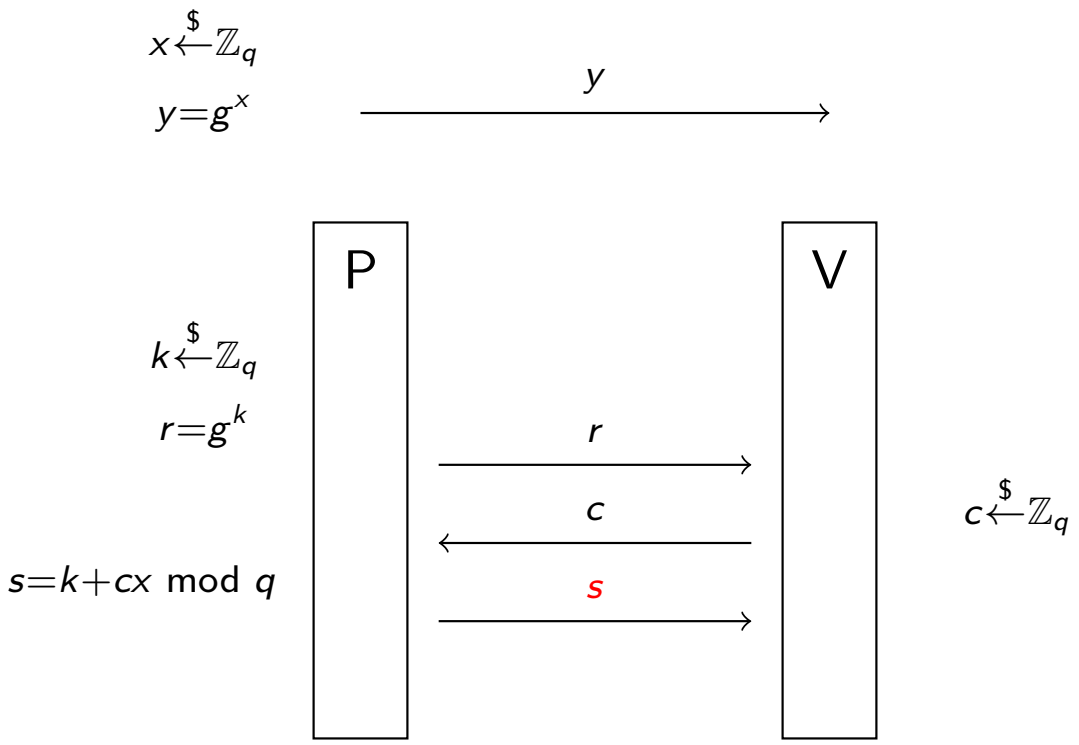
P sends $s = k + cx \bmod q$

V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.



Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$

V sends $c \xleftarrow{\$} \mathbb{Z}_q$

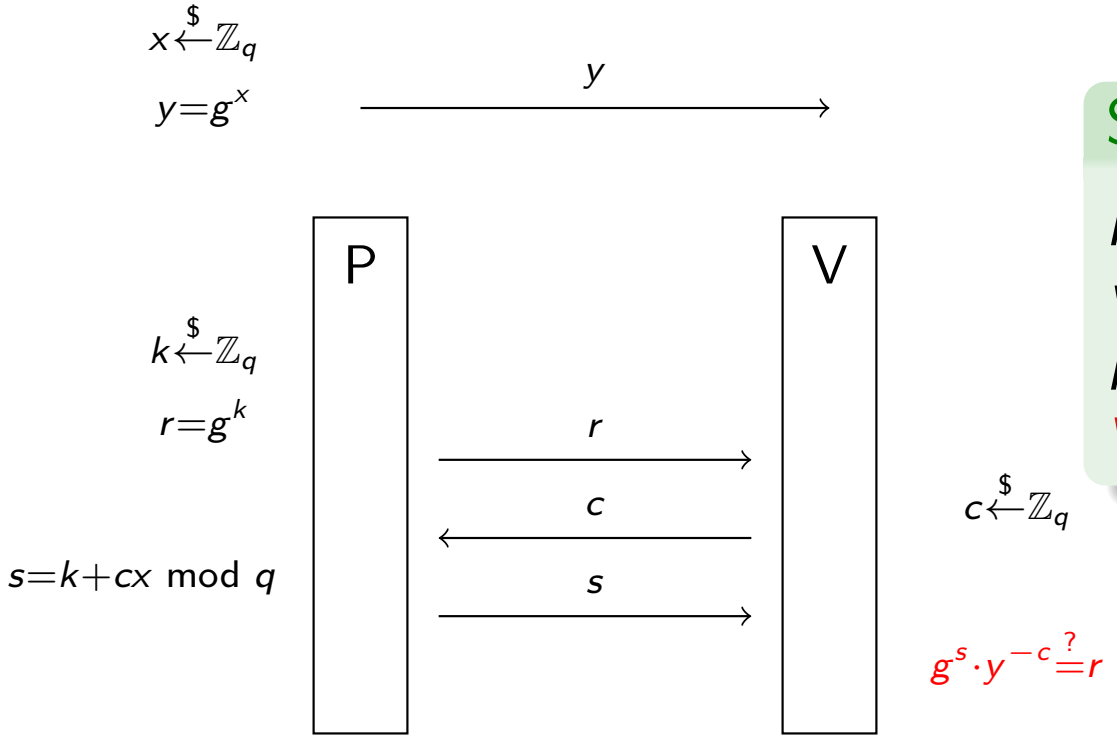
P sends $s = k + cx \bmod q$

V checks whether $g^s \cdot y^{-c} = r$

Schnorr's Protocol (1989)

Let $\mathbb{G} = \langle g \rangle$ be a group of prime order q

Prover P proves to verifier V that he knows the discrete log x of a public group element $y = g^x$. It is a 3-move protocol.



Scenario

P sends $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$

V sends $c \xleftarrow{\$} \mathbb{Z}_q$

P sends $s = k + cx \bmod q$

V checks whether $g^s \cdot y^{-c} = r$

The Fiat-Shamir heuristic

Fiat, Shamir (1986)
How to Prove Yourself: Practical Solutions to Identification and Signature Problems.
Advances in Cryptology - Crypto'86, Lect. Notes Comput. Science 263, pp. 186-194.

- In a 3-pass identification scheme, the messages are called **commitment**, **challenge** and **response**. The challenge is randomly chosen by V.

Fiat-Shamir Transform: replace the challenge by a hash value taken on scheme parameters and the commitment, thereby removing V. This transforms the protocol by making it *non-interactive*.

The intuition is that any "sufficiently random" hash function should preserve the security of the protocol.

The Fiat-Shamir heuristic

Fiat, Shamir (1986)
How to Prove Yourself: Practical Solutions to Identification and Signature Problems.
Advances in Cryptology - Crypto'86, Lect. Notes Comput. Science 263, pp. 186-194.

- In a 3-pass identification scheme, the messages are called **commitment**, **challenge** and **response**. The challenge is randomly chosen by V.

Fiat-Shamir Transform: replace the challenge by a hash value taken on scheme parameters and the commitment, thereby removing V. This transforms the protocol by making it *non-interactive*.

The intuition is that any "sufficiently random" hash function should preserve the security of the protocol.

Fiat-Shamir Identification Protocol

- **One-time setup:**
  - Trusted party T publishes an RSA modulus $N = pq$ but keeps primes p and q secret;
  - Alice select a secret $s \in \mathbb{Z}_n^*$ and publishes $v = s^2 \mod n$ as her public key.

- **Protocol:** (repeated ℓ times)
  - Alice picks $k \in \mathbb{Z}_n^*$ at random and sends $t = k^2 \mod n$ (commitment);
  - Bob picks at random a bit $c \in \{0, 1\}$ and sends it to Alice (challenge);
  - Alice computes $r = k \cdot s^c \mod n$ and sends it to Bob (response);
  - Bob accepts the proof if $r \neq 0$ and $r^2 = t \cdot v^c$.

(very similar to Guillou-Quisquater protocol – see TD)

Fiat Shamir signature

$\ell =$ **security parameter** (e.g. $\ell = 128$); $H : \{0,1\}^* \longrightarrow \{0,1\}^\ell$ hash function.

- **Key generation:**
  - Alice publishes an RSA modulus $N = pq$ but keeps primes p and q secret;
  - Alice select a secret $s \in \mathbb{Z}_n^*$, computes $v = s^2 \mod n$ and publishes (n, v) as her public key.

- **Signature generation:** message $m \in \{0,1\}$
  - Alice picks $k_1, \ldots, k_\ell \in \mathbb{Z}_n^*$ at random and computes $t_i = k_i^2 \mod n$ for $i \in [\![1, \ell]\!]$;
  - She computes $\vec{c} = (c_1, \ldots, c_\ell) = H(m, t_1, \ldots, t_\ell)$
  - Alice computes $r_i = k_i \cdot s^{c_i} \mod n$ for $i \in [\![1, \ell]\!]$ and the signature is $\sigma = (r_1, \ldots, r_n, \vec{c})$

- **Verification:**
  - Bob computes $\tilde{t}_i = r_i^2 v_i^{-c_i}$ for $i \in [\![1, \ell]\!]$ and accepts σ iff $\vec{c} = H(m, \tilde{t}_1, \ldots, \tilde{t}_n)$.

The Fiat-Shamir Transform

The same heuristic transformation can be applied to any 3-pass identification protocol to construct a signature scheme: the message m becomes a parameter of the hash value $e = H(t, m)$.

Intuitively, it should be hard for a forger to find m and a protocol transcript (t, e, s) for which it is true both that $e = H(t, m)$ and (t, e, s) is a valid transcript w.r.t. a given public key.

So-obtained signature schemes are **more efficient** (while hopefully achieving strong security) than other constructions.

The Fiat-Shamir Transform

The same heuristic transformation can be applied to any 3-pass identification protocol to construct a signature scheme: the message m becomes a parameter of the hash value $e = H(t, m)$.

Intuitively, it should be hard for a forger to find m and a protocol transcript (t, e, s) for which it is true both that $e = H(t, m)$ and (t, e, s) is a valid transcript w.r.t. a given public key.

So-obtained signature schemes are **more efficient** (while hopefully achieving strong security) than other constructions.

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.

P

Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.

P

V

Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.

$x \xleftarrow{\$} \mathbb{Z}_q$

P

V

Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.

$x \xleftarrow{\$} \mathbb{Z}_q$

$y = g^x$

P

V

Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

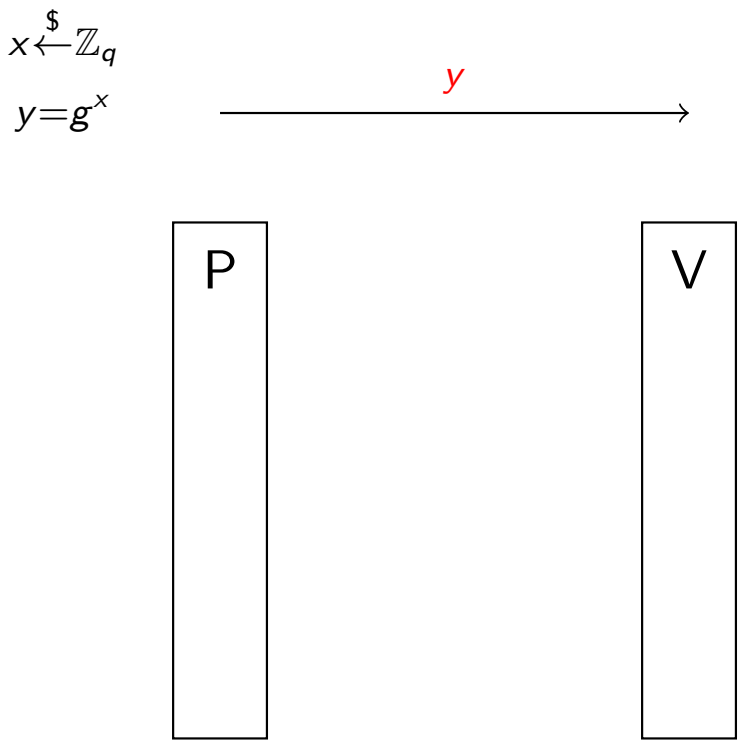
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

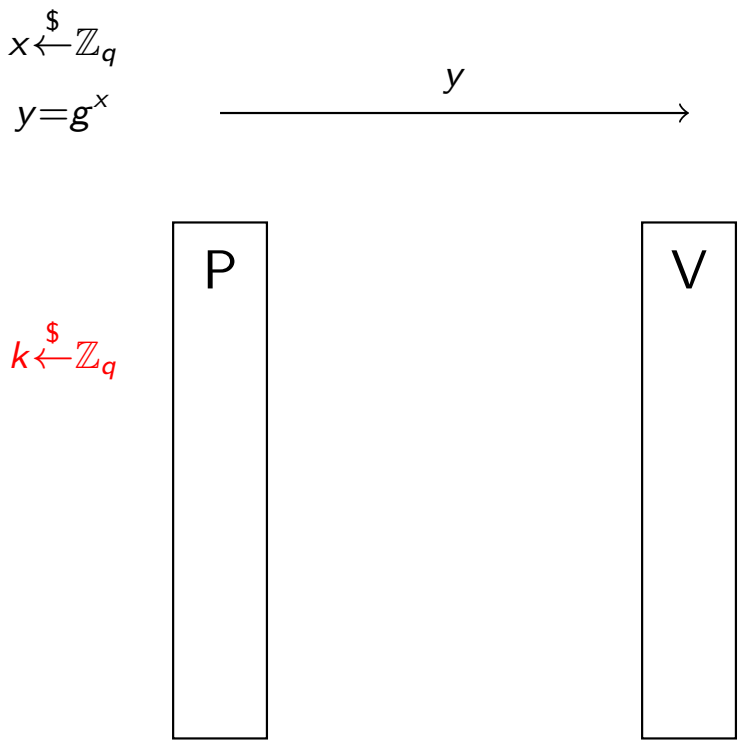
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

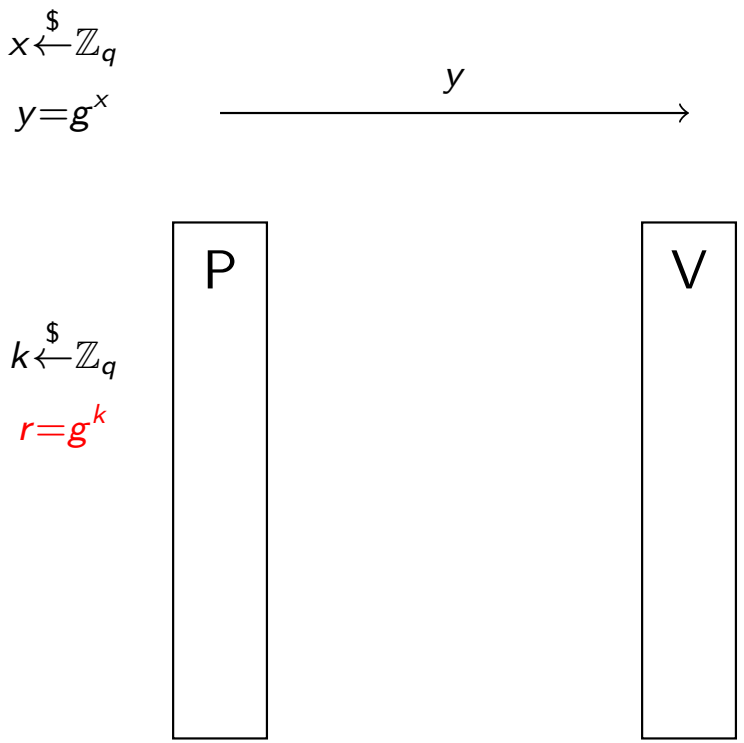
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

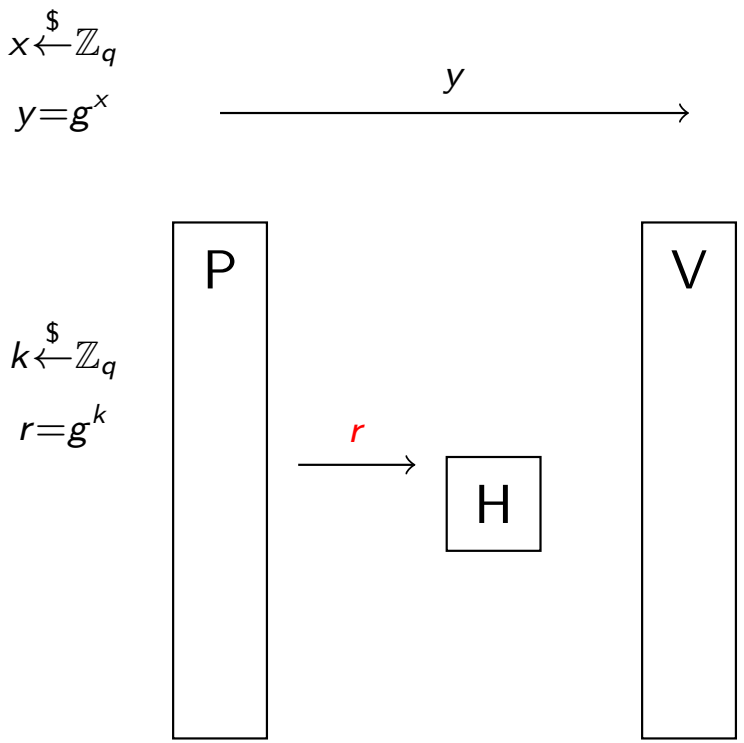
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

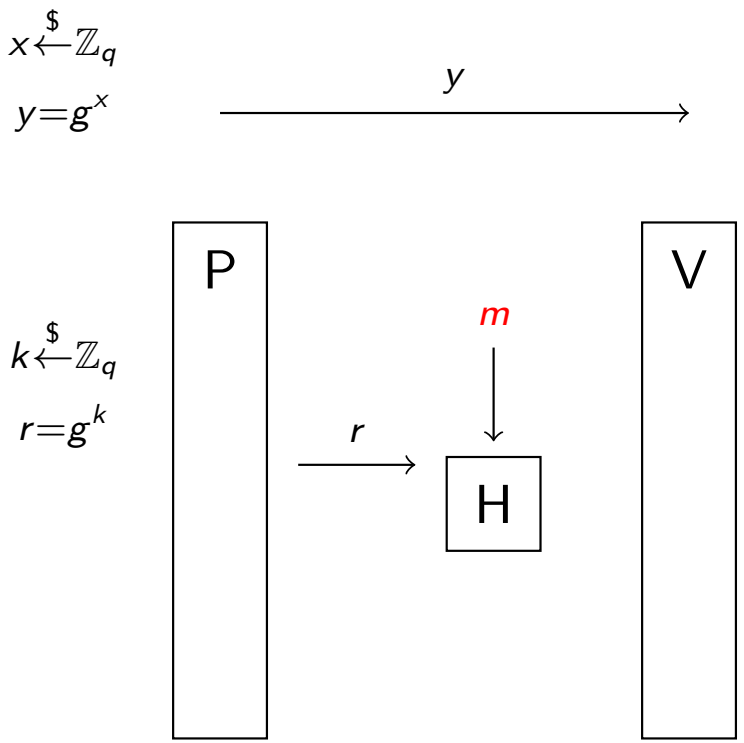
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

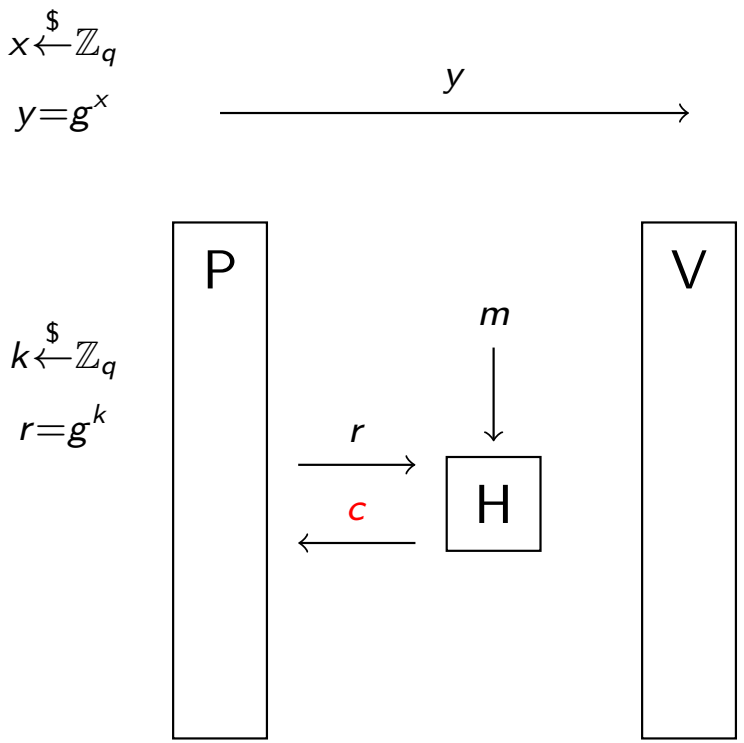
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

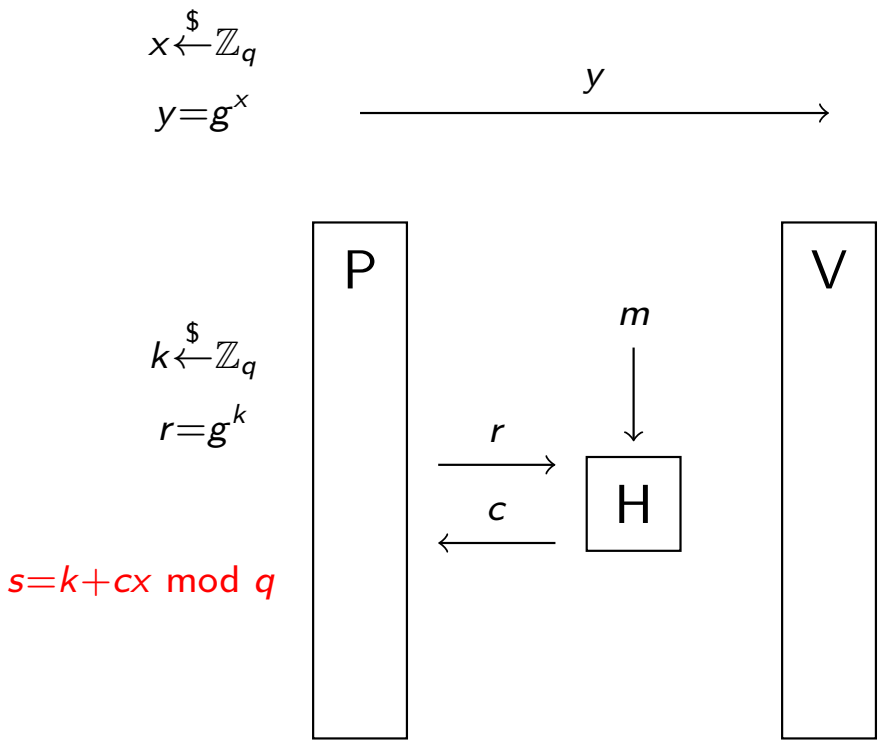
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

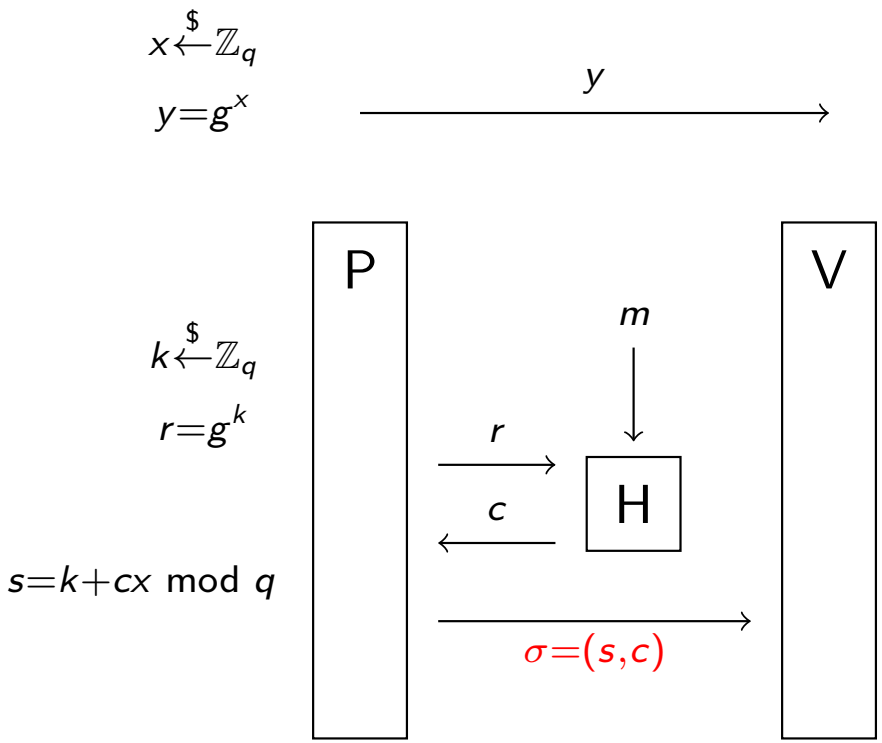
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

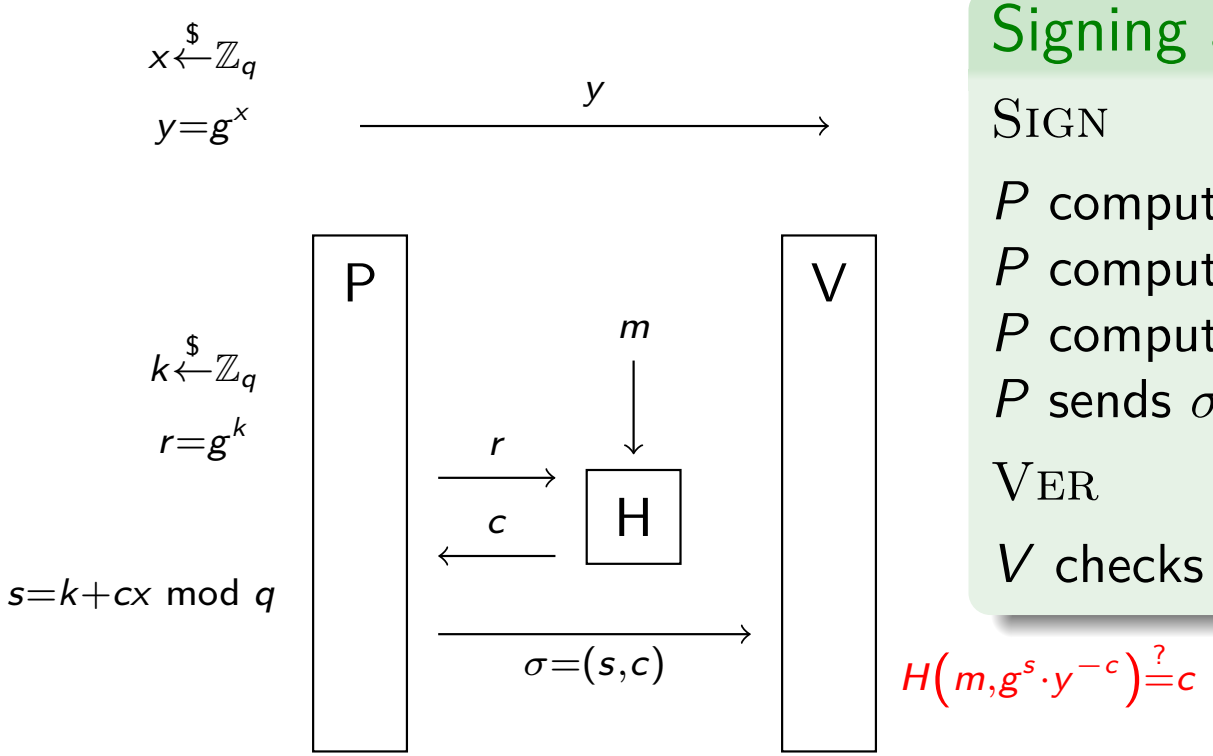
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

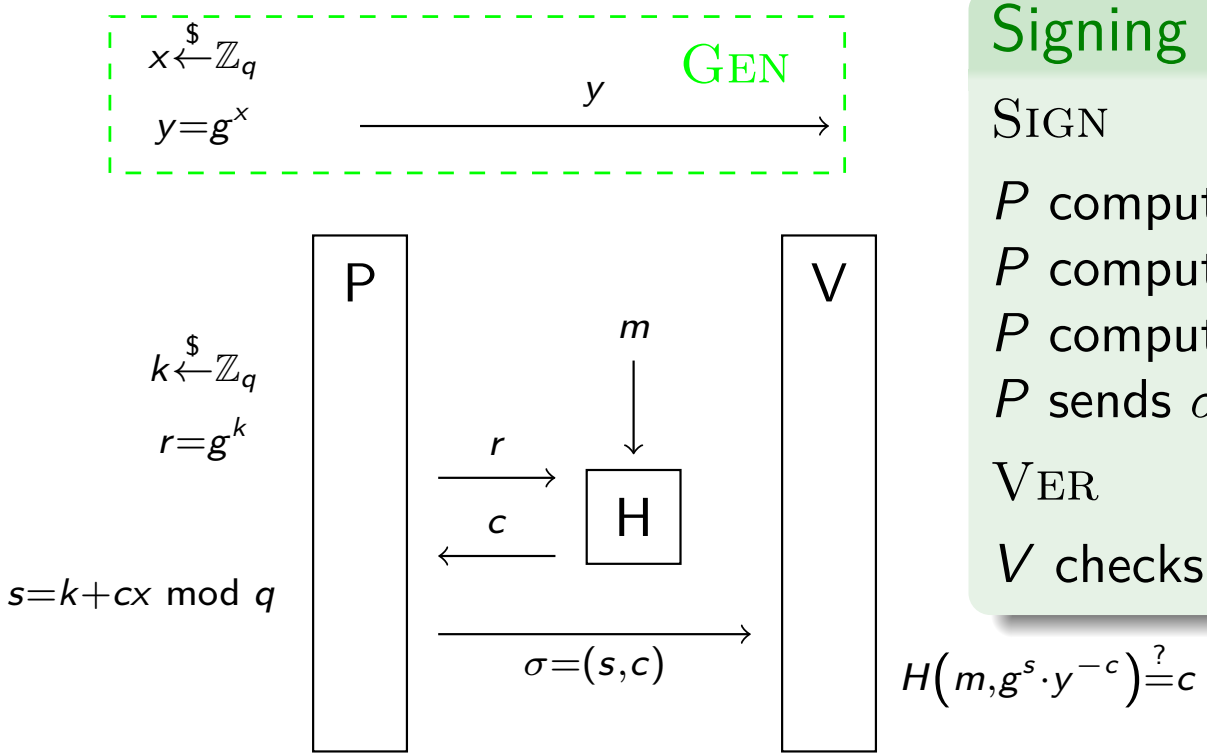
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

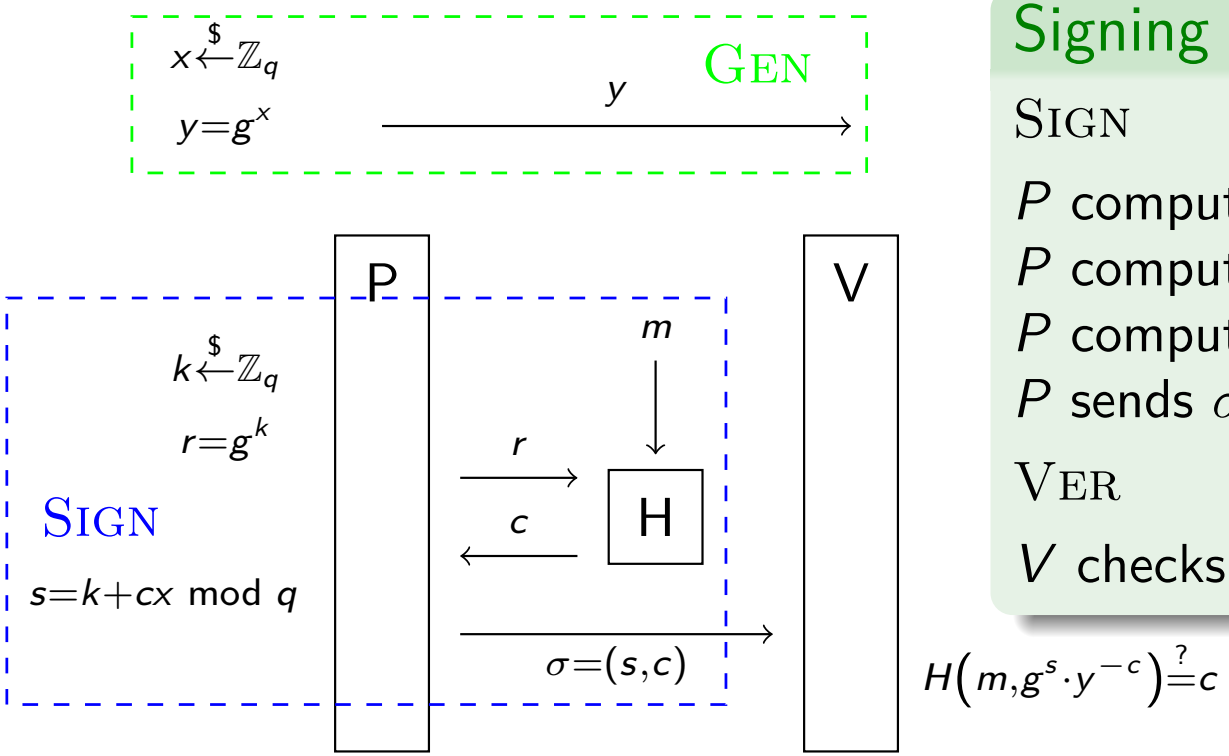
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

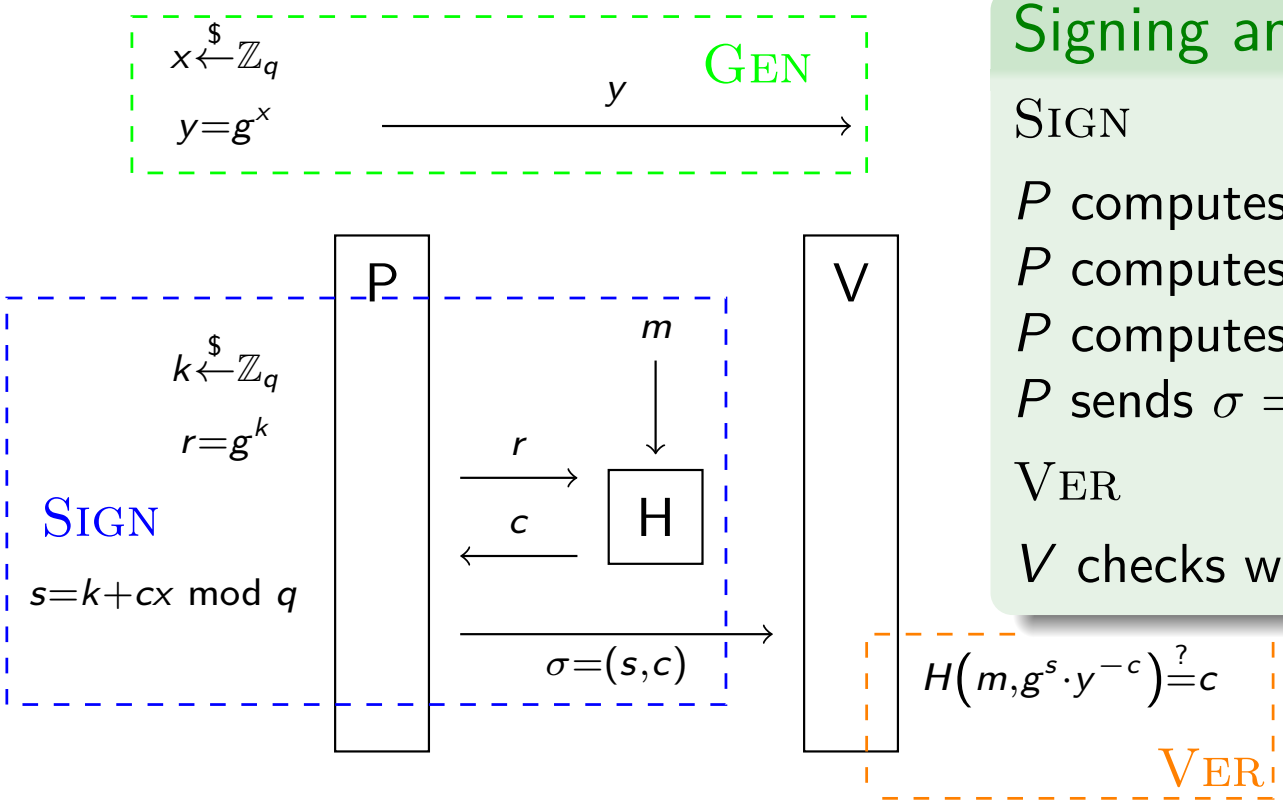
$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Schnorr Signatures (via the Fiat-Shamir Transform)

Introduce a hash function $H : \{0,1\}^\star \mapsto \mathbb{Z}_q$

Schnorr's signature scheme is a tuple of probabilistic algorithms $(\textsc{Gen}, \textsc{Sign}, \textsc{Ver})$ defined as follows.



Signing and Verifying

$\textsc{Sign}$

P computes $r = g^k$ where $k \xleftarrow{\$} \mathbb{Z}_q$
P computes $c = H(m, r)$
P computes $s = k + cx \bmod q$
P sends $\sigma = (s, c)$

$\textsc{Ver}$

V checks whether $H(m, g^s \cdot y^{-c}) = c$

Outline

1. Fiat-Shamir heuristic and Schnorr Signatures
   - Schnorr's Protocol
   - The Fiat-Shamir heuristic
   - Schnorr signatures
2. Secret-Sharing
   - Motivation
   - In the "Real-World"
   - Shamir's Secret Sharing
3. Multi-Party Computation
   - Motivation
   - Example: Mean computation
   - BGW Protocol – Semi-Honest Security

Key storage: problems

- Reliability and confidentiality of important data:
  - Information can be secured by encryption
  - After that, many copies of the ciphertext can be made
- How to secure the secret key?
  - Encrypting of key – vicious cycle
  - Replicating key – insecure
- **Idea:** Distribute the key to a group, s.t. nobody by itself knows it (entirely)

Secret Sharing: More Motivations

- USSR: At least two of the three nuclear buttons must have been pressed **simultaneously**
- Any other process where you might not trust a single authority
- Threshold cryptography, multi-party computation
- Computation can be performed in a distributed way by "trusted" subsets of parties

Secret Sharing: More Motivations

- USSR: At least two of the three nuclear buttons must have been pressed **simultaneously**
- Any other process where you might not trust a single authority
- Threshold cryptography, multi-party computation
- Computation can be performed in a distributed way by "trusted" subsets of parties

Secret Sharing: More Motivations

- USSR: At least two of the three nuclear buttons must have been pressed **simultaneously**
- Any other process where you might not trust a single authority
- Threshold cryptography, multi-party computation
- Computation can be performed in a distributed way by "trusted" subsets of parties

n-out-of-n Secret Sharing

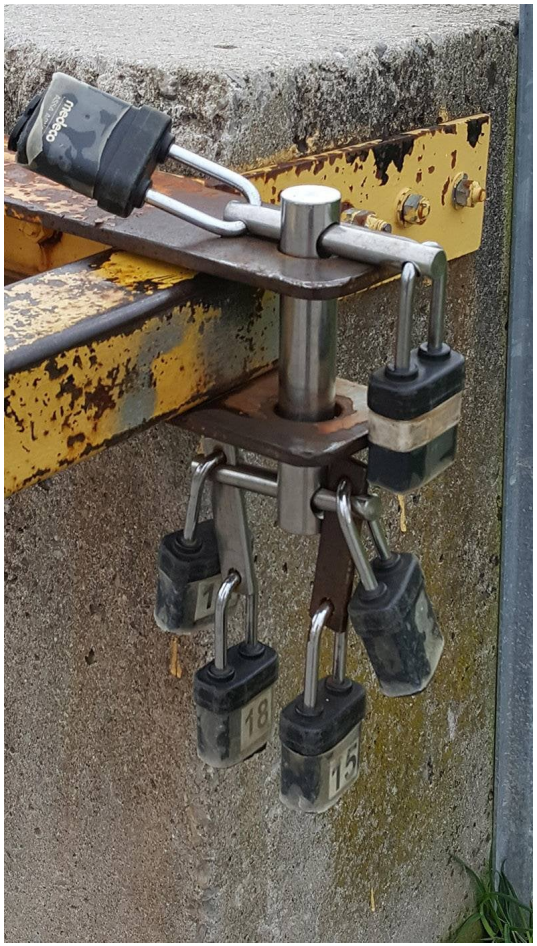
n-out-of-n Secret Sharing

1-out-of-n Secret Sharing

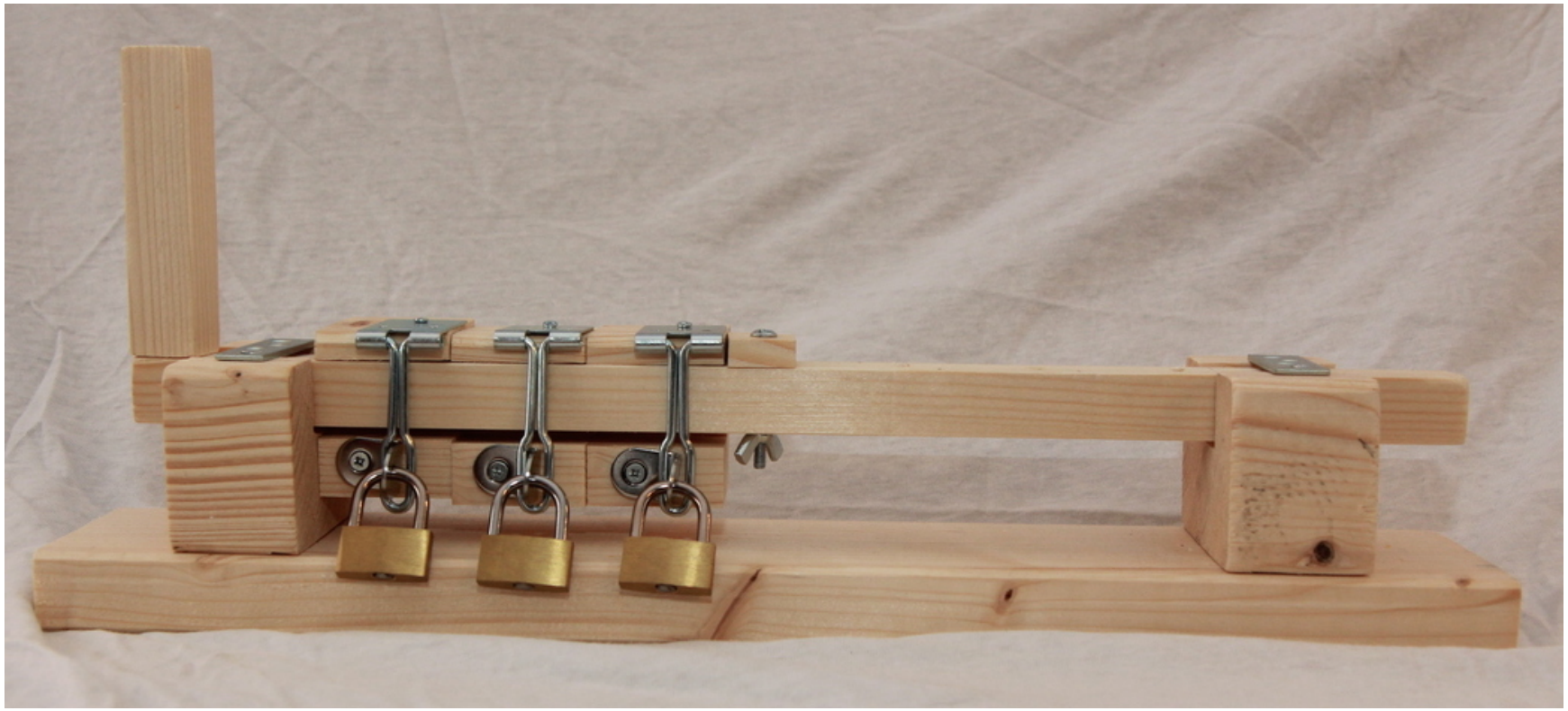
1-out-of-n Secret Sharing

1-out-of-n Secret Sharing

1-out-of-n Secret Sharing

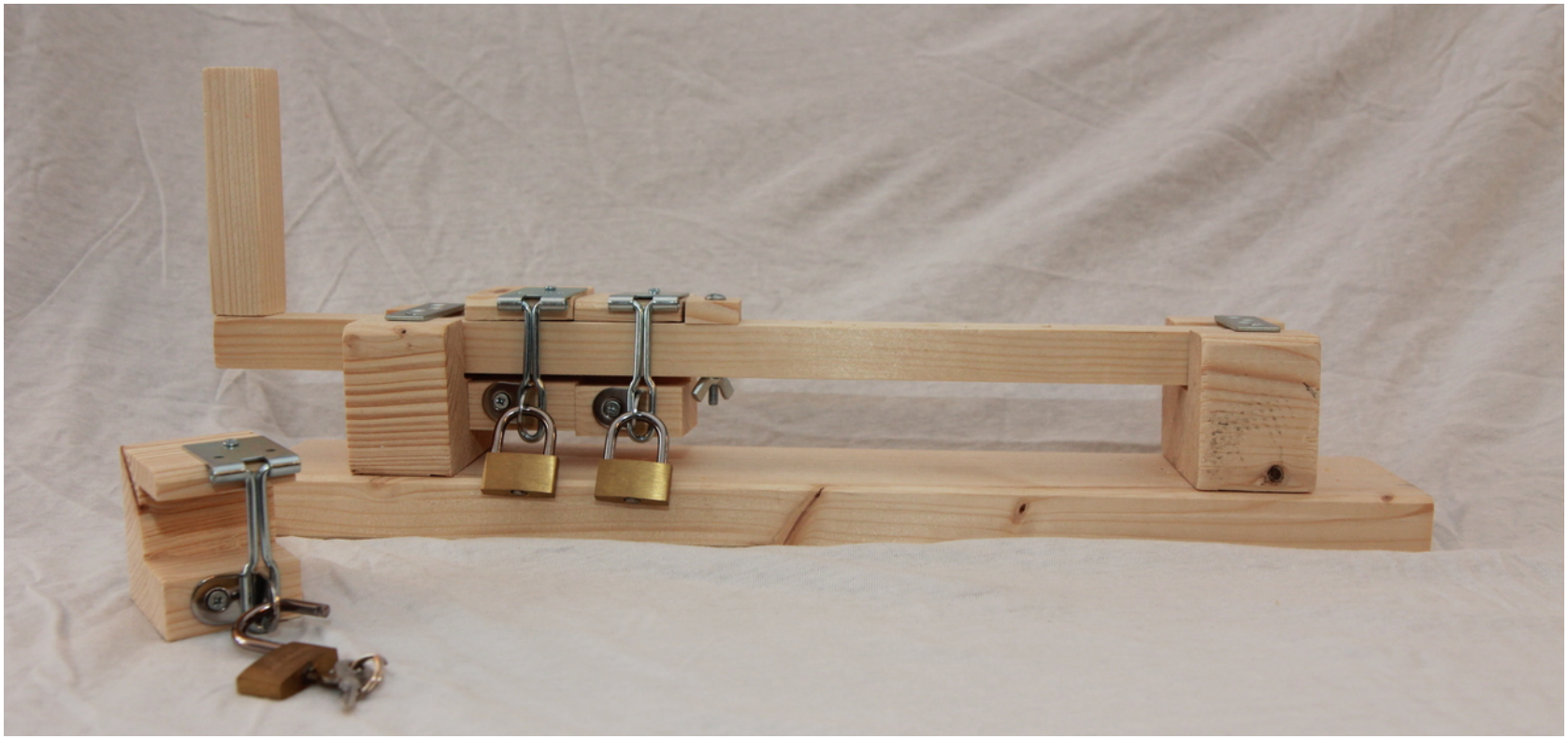


2-out-of-n Secret Sharing



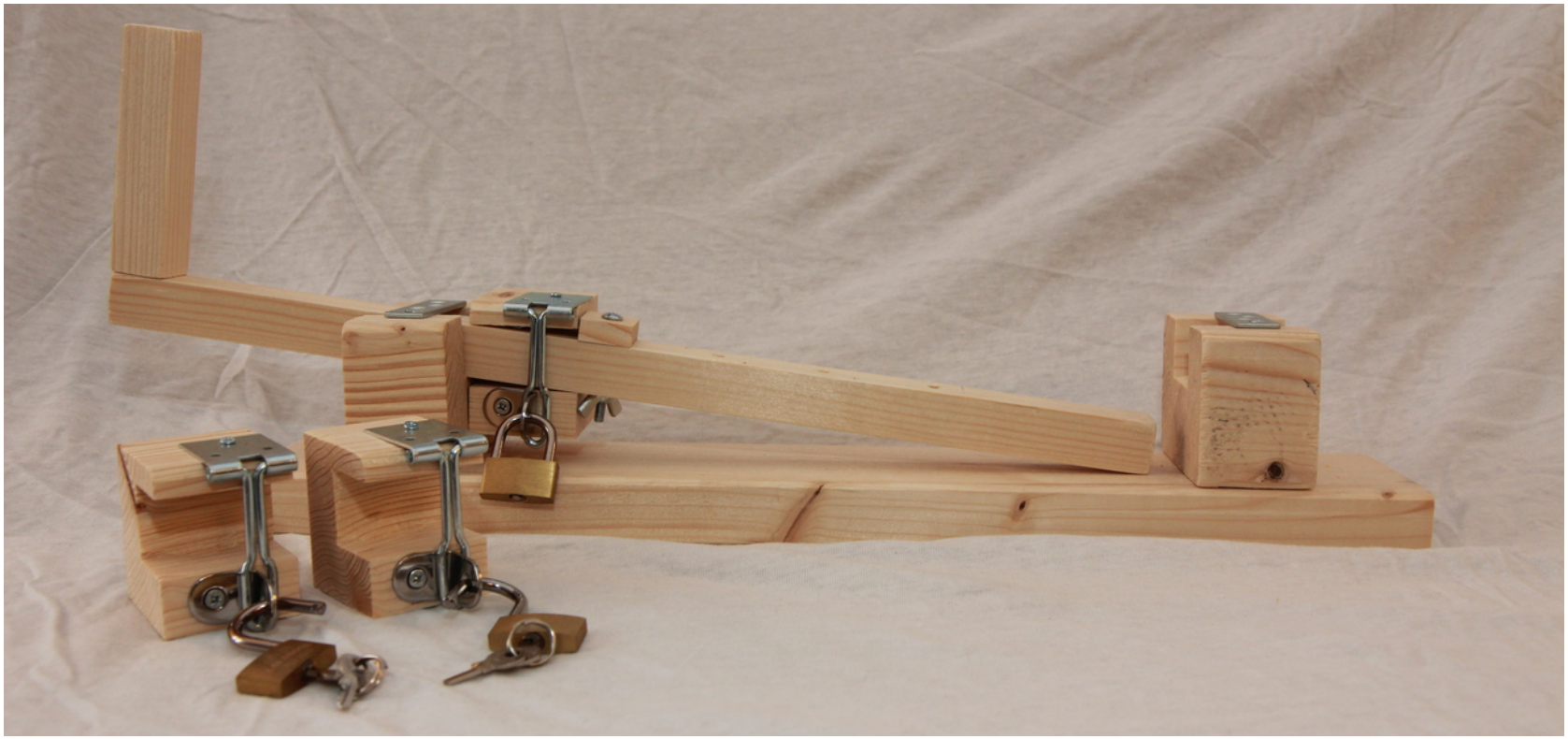
J. Dreier, J.-G. Dumas, P. Lafourcade, L. Robert
Optimal threshold padlock systems.
J. Comput. Secur. 30(5): 655-688 (2022)

2-out-of-n Secret Sharing



J. Dreier, J.-G. Dumas, P. Lafourcade, L. Robert
Optimal threshold padlock systems.
J. Comput. Secur. 30(5): 655-688 (2022)

2-out-of-n Secret Sharing



J. Dreier, J.-G. Dumas, P. Lafourcade, L. Robert
Optimal threshold padlock systems.
J. Comput. Secur. 30(5): 655-688 (2022)

Secret sharing schemes: Definition

- A dealer shares a secret s between n parties
- Each party P_i for $i \in \{1, \ldots, n\}$ receives a share s_i
- **Predefined groups** of participants can cooperate to reconstruct the secret
- Smaller subgroups cannot get any information about the secret

Example (bad)

- Let k be a 128-bit block cipher key.
  Share it between 4 parties giving to all parties 32 bits of the key
- the **whole** group can cooperate to reconstruct the secret k
- Why is this bad? The second requirement is violated

Ciphertext-only attack: Three participants can recover the plaintext by themselves, by doing a 2^{32}-**time** exhaustive search

Example (bad)

- Let k be a 128-bit block cipher key.
  Share it between 4 parties giving to all parties 32 bits of the key
- the **whole** group can cooperate to reconstruct the secret k
- Why is this bad? The second requirement is violated

Ciphertext-only attack: Three participants can recover the plaintext by themselves, by doing a 2^{32}-**time** exhaustive search

Example (bad)

- Let k be a 128-bit block cipher key.

  Share it between 4 parties giving to all parties 32 bits of the key

- the **whole** group can cooperate to reconstruct the secret k

- Why is this bad? The second requirement is violated

  Ciphertext-only attack: Three participants can recover the plaintext by themselves, by doing a 2^{32}-**time** exhaustive search

k-out-of-n Secret sharing schemes: Definition

- A dealer shares a secret s among n parties
- Each party P_i for $i \in \{1, \ldots, n\}$ receives a share s_i
- A group of any k participants can cooperate to reconstruct the secret
- **No group** of $k-1$ participants can get any information about the secret
- Threshold Access Structure

n-out-of-n Secret Sharing

- Let s be a secret from group $\mathbb{G}$.
  Examples: $\mathbb{G} = (\mathbb{Z}/n\mathbb{Z}, +)$, $\mathbb{G} = (\{0,1\}^n, \oplus)$
- Dealer chooses uniformly random $s_1, \ldots, s_{n-1}$ in $\mathbb{G}$ and computes

$$s_n = s - (s_1 + \cdots + s_{n-1})$$

  The **shares** are $(s_1, \ldots, s_n)$
- Given $(s_1, \ldots, s_n)$, one can successfully recover $s = s_1 + \cdots + s_n$
- Given all s_i's for $i \neq j$,

$$\sum_{i \neq j} s_i = s - s_j$$

  is random $\rightsquigarrow$ no information about s

n-out-of-n Secret Sharing

- Let s be a secret from group $\mathbb{G}$.
  Examples: $\mathbb{G} = (\mathbb{Z}/n\mathbb{Z}, +)$, $\mathbb{G} = (\{0,1\}^n, \oplus)$
- Dealer chooses uniformly random $s_1, \ldots, s_{n-1}$ in $\mathbb{G}$ and computes

$$s_n = s - (s_1 + \cdots + s_{n-1})$$

  The **shares** are $(s_1, \ldots, s_n)$
- Given $(s_1, \ldots, s_n)$, one can successfully recover $s = s_1 + \cdots + s_n$
- Given all s_i's for $i \neq j$,

$$\sum_{i \neq j} s_i = s - s_j$$

  is random $\rightsquigarrow$ no information about s

n-out-of-n Secret Sharing

- Let s be a secret from group $\mathbb{G}$.
  Examples: $\mathbb{G} = (\mathbb{Z}/n\mathbb{Z}, +)$, $\mathbb{G} = (\{0,1\}^n, \oplus)$
- Dealer chooses uniformly random $s_1, \ldots, s_{n-1}$ in $\mathbb{G}$ and computes

$$s_n = s - (s_1 + \cdots + s_{n-1})$$

  The **shares** are $(s_1, \ldots, s_n)$
- Given $(s_1, \ldots, s_n)$, one can successfully recover $s = s_1 + \cdots + s_n$
- Given all s_i's for $i \neq j$,

$$\sum_{i \neq j} s_i = s - s_j$$

  is random $\rightsquigarrow$ no information about s

n-out-of-n Secret Sharing

- Let s be a secret from group $\mathbb{G}$.
  Examples: $\mathbb{G} = (\mathbb{Z}/n\mathbb{Z}, +)$, $\mathbb{G} = (\{0,1\}^n, \oplus)$
- Dealer chooses uniformly random $s_1, \ldots, s_{n-1}$ in $\mathbb{G}$ and computes

$$s_n = s - (s_1 + \cdots + s_{n-1})$$

  The **shares** are $(s_1, \ldots, s_n)$
- Given $(s_1, \ldots, s_n)$, one can successfully recover $s = s_1 + \cdots + s_n$
- Given all s_i's for $i \neq j$,

$$\sum_{i \neq j} s_i = s - s_j$$

  is random $\rightsquigarrow$ no information about s

Shamir's k-out-of-n Threshold Secret Sharing

Mathematical basis:

- Given k points on the plane $(x_1, y_1), \ldots, (x_k, y_k)$, all x_i distinct, there exists a unique polynomial P of degree $k-1$, s.t.

$$P(x_i) = y_i \text{ for all } i \in \{1, \ldots, k\}$$

- Constructive proof: Given these k points, one can recover P by using the **Lagrange interpolation formula**
- This holds also in any field and in particular in finite fields such as $\mathbb{Z}/p\mathbb{Z}$ with p prime

Shamir's k-out-of-n Threshold Secret Sharing

Mathematical basis:

- Given k points on the plane $(x_1, y_1), \ldots, (x_k, y_k)$, all x_i distinct, there exists a unique polynomial P of degree $k-1$, s.t.

$$P(x_i) = y_i \text{ for all } i \in \{1, \ldots, k\}$$

- Constructive proof: Given these k points, one can recover P by using the **Lagrange interpolation formula**
- This holds also in any field and in particular in finite fields such as $\mathbb{Z}/p\mathbb{Z}$ with p prime

Shamir's k-out-of-n Threshold Secret Sharing

Mathematical basis:

- Given k points on the plane $(x_1, y_1), \ldots, (x_k, y_k)$, all x_i distinct, there exists a unique polynomial P of degree $k-1$, s.t.

$$P(x_i) = y_i \text{ for all } i \in \{1, \ldots, k\}$$

- Constructive proof: Given these k points, one can recover P by using the **Lagrange interpolation formula**
- This holds also in any field and in particular in finite fields such as $\mathbb{Z}/p\mathbb{Z}$ with p prime

Shamir's k-out-of-n Threshold Secret Sharing

Description. Dealing phase:

- Let s be a secret from some $\mathbb{Z}/p\mathbb{Z}$ with p prime with $p > n$
- Select a random polynomial

$$P(x) = p_0 + p_1 x + p_2 x^2 + \cdot + p_{k-1} x^{k-1}$$

  under the condition that $P(0) = s$:
  - Select $p_1, \ldots, p_{k-1}$ in $\mathbb{Z}/p\mathbb{Z}$ uniformly at random
  - Set $p_0 = s$
- For $i \in \{1, \ldots, n\}$, give the **share** $s_i = (i, P(i))$ to the i-th party

Shamir's k-out-of-n Threshold Secret Sharing

Description. Dealing phase:

- Let s be a secret from some $\mathbb{Z}/p\mathbb{Z}$ with p prime with $p > n$
- Select a random polynomial

$$P(x) = p_0 + p_1 x + p_2 x^2 + \cdot + p_{k-1} x^{k-1}$$

  under the condition that $P(0) = s$:
  - Select $p_1, \ldots, p_{k-1}$ in $\mathbb{Z}/p\mathbb{Z}$ uniformly at random
  - Set $p_0 = s$
- For $i \in \{1, \ldots, n\}$, give the **share** $s_i = (i, P(i))$ to the i-th party

Shamir's k-out-of-n Threshold Secret Sharing

Description. Dealing phase:

- Let s be a secret from some $\mathbb{Z}/p\mathbb{Z}$ with p prime with $p > n$
- Select a random polynomial

$$P(x) = p_0 + p_1 x + p_2 x^2 + \cdot + p_{k-1} x^{k-1}$$

  under the condition that $P(0) = s$:
  - Select $p_1, \ldots, p_{k-1}$ in $\mathbb{Z}/p\mathbb{Z}$ uniformly at random
  - Set $p_0 = s$
- For $i \in \{1, \ldots, n\}$, give the **share** $s_i = (i, P(i))$ to the i-th party

Lagrange Interpolation (1/2)

Given, $x_1, \ldots, x_k$ distinct points in a field $\mathbb{F}$. Consider

$$\ell_j(x) := \prod_{\substack{1 \le m \le k \\ m \ne j}} \frac{x - x_m}{x_j - x_m}$$

$$= \frac{(x - x_1)}{(x_j - x_1)} \cdots \frac{(x - x_{j-1})}{(x_j - x_{j-1})} \frac{(x - x_{j+1})}{(x_j - x_{j+1})} \cdots \frac{(x - x_k)}{(x_j - x_k)},$$

we have for all $(j \ne i)$

$$\ell_j(x_i) = \prod_{m \ne j} \frac{x_i - x_m}{x_j - x_m} = \frac{(x_i - x_1)}{(x_j - x_1)} \cdots \frac{(x_i - x_i)}{(x_j - x_i)} \cdots \frac{(x_i - x_k)}{(x_j - x_k)} = 0.$$

and

$$\ell_j(x_j) := \prod_{m \ne j} \frac{x_j - x_m}{x_j - x_m} = 1$$

Lagrange Interpolation (2/2)

Given $(y_1, \ldots, y_k)$ in $\mathbb{F}^k$. Consider

$$P(x) := \sum_{j=1}^{k} y_j \ell_j(x)$$

We have

$$P(x_i) = \sum_{j=1}^{k} y_j \ell_j(x_i) = \sum_{j=1}^{k} y_j \delta_{ji} = y_i.$$

where

$$\ell_j(x_i) = \delta_{ji} = \begin{cases} 1, & \text{if } j = i \\ 0, & \text{if } j \neq i \end{cases},$$

Thus the function $P(x)$ is a polynomial with degree **at most** k and where $P(x_i) = y_i$ for all $i \in \{1, \ldots, k\}$. Moreover, this polynomial is unique

Shamir's k-out-of-n Threshold Secret Sharing

Theorem

The secret s can be reconstucted from every subset of k shares.

Proof. By the Lagrange formula, given k points (x_i, y_i) for $i \in \{1, \ldots, k\}$:

$$P(x) := \sum_{j=0}^{k} y_j \prod_{\substack{0 \leq m \leq k \\ m \neq j}} \frac{x - x_m}{x_j - x_m}$$

and thus

$$s = P(0) = \sum_{j=0}^{k} y_j \prod_{\substack{0 \leq m \leq k \\ m \neq j}} \frac{-x_m}{x_j - x_m}$$

$\square$

Shamir's k-out-of-n Threshold Secret Sharing

Theorem

Any subset of up to $k-1$ shares does not leak any information on the secret.

Proof: Given $k-1$ shares (x_i, y_i) for $i \in \{1, \ldots, k-1\}$, every candidate secret $s^* \in \mathbb{Z}/p\mathbb{Z}$ corresponds to an unique polynomial of degree at most $k-1$ for which $P(0) = s^*$.

From the construction of polynomials, for all $s^* \in \mathbb{Z}/p\mathbb{Z}$, probabilities $\Pr[s = s^*]$ are equal. $\square$

Conclusion: Shamir's scheme is perfectly secure and does not depend on the computational power of any party.

Shamir's k-out-of-n Threshold Secret Sharing

Theorem

Any subset of up to $k-1$ shares does not leak any information on the secret.

Proof: Given $k-1$ shares (x_i, y_i) for $i \in \{1, \ldots, k-1\}$, every candidate secret $s^* \in \mathbb{Z}/p\mathbb{Z}$ corresponds to an unique polynomial of degree at most $k-1$ for which $P(0) = s^*$.

From the construction of polynomials, for all $s^* \in \mathbb{Z}/p\mathbb{Z}$, probabilities $\Pr[s = s^*]$ are equal. $\square$

Conclusion: Shamir's scheme is perfectly secure and does not depend on the computational power of any party.

Shamir's k-out-of-n Threshold Secret Sharing

Theorem

Any subset of up to $k-1$ shares does not leak any information on the secret.

Proof: Given $k-1$ shares (x_i, y_i) for $i \in \{1, \ldots, k-1\}$, every candidate secret $s^* \in \mathbb{Z}/p\mathbb{Z}$ corresponds to an unique polynomial of degree at most $k-1$ for which $P(0) = s^*$.

From the construction of polynomials, for all $s^* \in \mathbb{Z}/p\mathbb{Z}$, probabilities $\Pr[s = s^*]$ are equal. $\square$

Conclusion: Shamir's scheme is perfectly secure and does not depend on the computational power of any party.

Shamir's Secret Sharing - Illustration

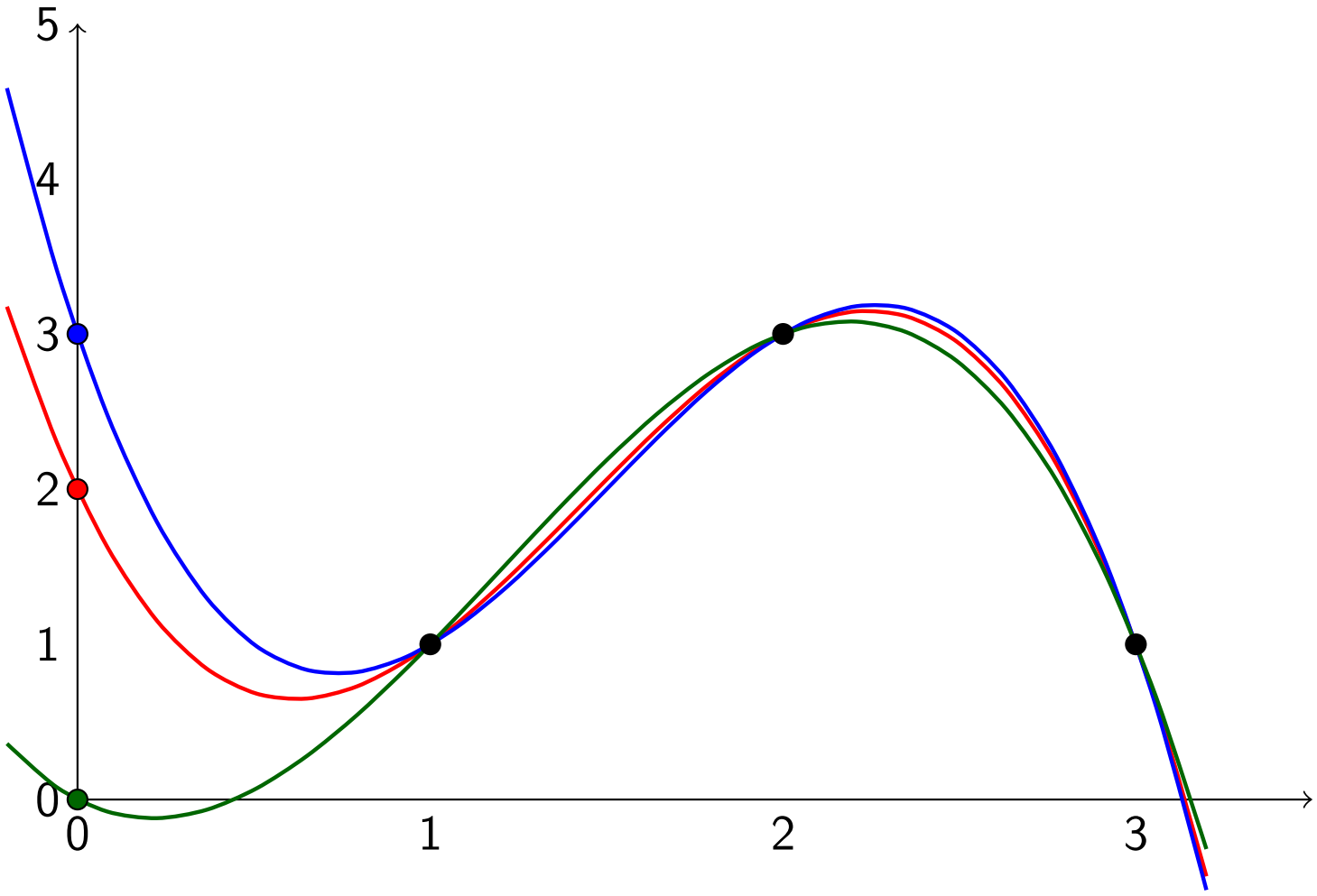


Table des matières

1. Fiat-Shamir heuristic and Schnorr Signatures
   - Schnorr's Protocol
   - The Fiat-Shamir heuristic
   - Schnorr signatures
2. Secret-Sharing
   - Motivation
   - In the "Real-World"
   - Shamir's Secret Sharing
3. Multi-Party Computation
   - Motivation
   - Example: Mean computation
   - BGW Protocol – Semi-Honest Security

Multi-Party Computation

- a set of parties with private inputs
- wish to jointly compute a function of their inputs so that certain security properties are preserved

- **Examples**
  - secure elections
  - auctions
  - privacy preserving data mining
  - ...

- **Security level**
  - computational security $\rightsquigarrow$ computational assumptions
  - unconditional security $\rightsquigarrow$ information-theoretically secure

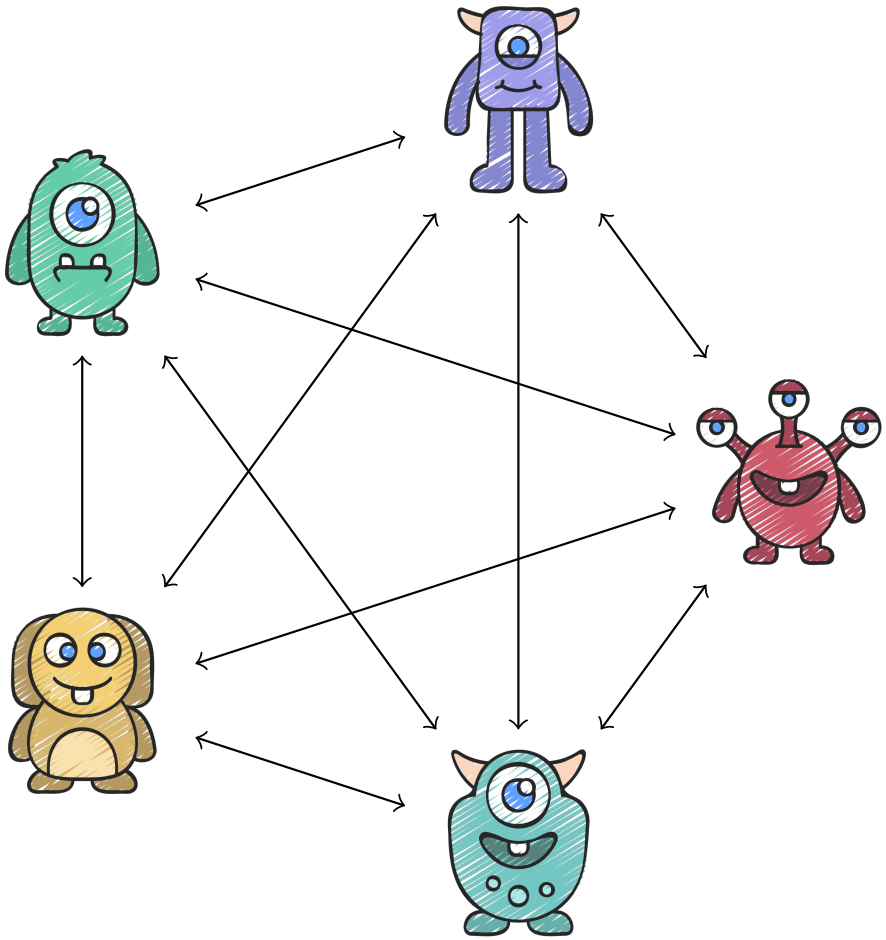
Multi-Party Computation

- a set of parties with private inputs
- wish to jointly compute a function of their inputs so that certain security properties are preserved

- **Examples**
  - secure elections
  - auctions
  - privacy preserving data mining
  - ...

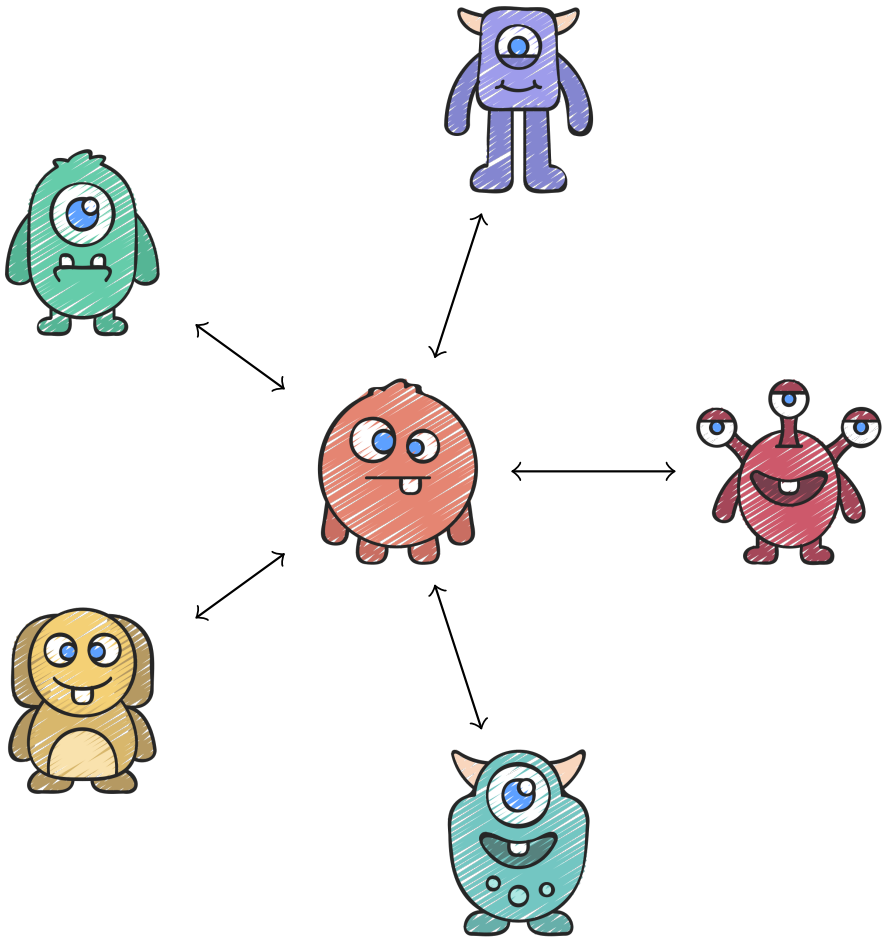
- **Security level**
  - computational security $\rightsquigarrow$ computational assumptions
  - unconditional security $\rightsquigarrow$ information-theoretically secure

Multi-Party Computation



- computation between parties who do not trust each other
- preserve the privacy of each player's inputs
- guarantee the correctness of the computation

Multi-Party Computation



- Trusted party: these tasks can be easily computed by a trusted third party
- Security : Same guarantees without involving a trusted third party

Multi-Party Computation

- Set of parties $P_1, \ldots, P_n$
- Each holds some private input $x_1, \ldots, x_n$
- The parties wish to compute a joint function $f(x_1, \ldots, x_n)$ while keeping their inputs private

- Some parties might be corrupted:
  - Semi-honest: Follow the protocol specifications' but try to gain some extra information by pooling their views
  - Malicious: Might act arbitrarily

- **Correctness:** The output of the parties is $f(x_1, \ldots, x_n)$
- **Privacy:** The corrupted parties do not learn anything about the honest parties' inputs
- **Guaranteed output delivery:** The adversary should not prevent the honest parties from obtaining output

Multi-Party Computation

- Set of parties $P_1, \ldots, P_n$
- Each holds some private input $x_1, \ldots, x_n$
- The parties wish to compute a joint function $f(x_1, \ldots, x_n)$ while keeping their inputs private
- Some parties might be corrupted:
  - Semi-honest: Follow the protocol specifications' but try to gain some extra information by pooling their views
  - Malicious: Might act arbitrarily
- **Correctness:** The output of the parties is $f(x_1, \ldots, x_n)$
- **Privacy:** The corrupted parties do not learn anything about the honest parties' inputs
- **Guaranteed output delivery:** The adversary should not prevent the honest parties from obtaining output

Multi-Party Computation

- Set of parties $P_1, \ldots, P_n$
- Each holds some private input $x_1, \ldots, x_n$
- The parties wish to compute a joint function $f(x_1, \ldots, x_n)$ while keeping their inputs private
- Some parties might be corrupted:
  - Semi-honest: Follow the protocol specifications' but try to gain some extra information by pooling their views
  - Malicious: Might act arbitrarily
- **Correctness:** The output of the parties is $f(x_1, \ldots, x_n)$
- **Privacy:** The corrupted parties do not learn anything about the honest parties' inputs
- **Guaranteed output delivery:** The adversary should not prevent the honest parties from obtaining output

Multi-Party Computation

Semi-Honest Adversaries

For every n-ary function $f(x_1, \ldots, x_n)$, there exists a protocol for computing with perfect security in the presence of a semi-honest adversary controlling $t < n/2$ parties

Malicious Adversaries

For every n-ary function $f(x_1, \ldots, x_n)$, there exists a protocol for computing with perfect security in the presence of a malicious adversary controlling $t < n/3$ parties

Multi-Party Computation

Semi-Honest Adversaries

For every n-ary function $f(x_1, \ldots, x_n)$, there exists a protocol for computing with perfect security in the presence of a semi-honest adversary controlling $t < n/2$ parties

Malicious Adversaries

For every n-ary function $f(x_1, \ldots, x_n)$, there exists a protocol for computing with perfect security in the presence of a malicious adversary controlling $t < n/3$ parties

Multi-Party Computation

Semi-Honest Adversaries

For every n-ary function $f(x_1, \ldots, x_n)$, there exists a protocol for computing with perfect security in the presence of a semi-honest adversary controlling $t < n/2$ parties

Malicious Adversaries

For every n-ary function $f(x_1, \ldots, x_n)$, there exists a protocol for computing with perfect security in the presence of a malicious adversary controlling $t < n/3$ parties

Mean computation

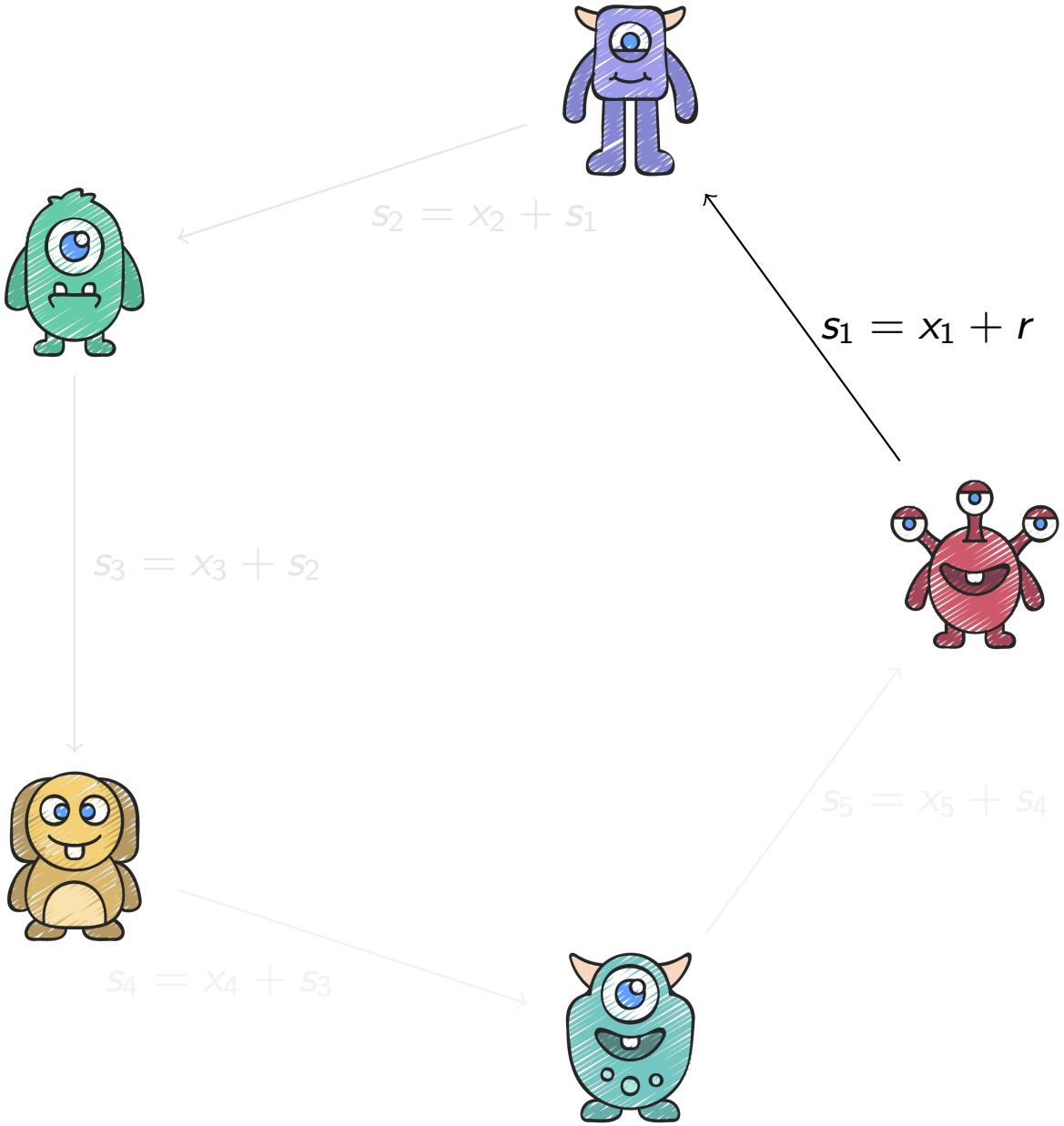
- n players $P_1, \ldots, P_n$ with inputs $x_1, \ldots, x_n$ integers in $\{1, \ldots, B\}$
- Consider an integer q such that $q > nB$.

- P_1 picks r **uniformly at random** in $\mathbb{Z}/q\mathbb{Z}$ and sends

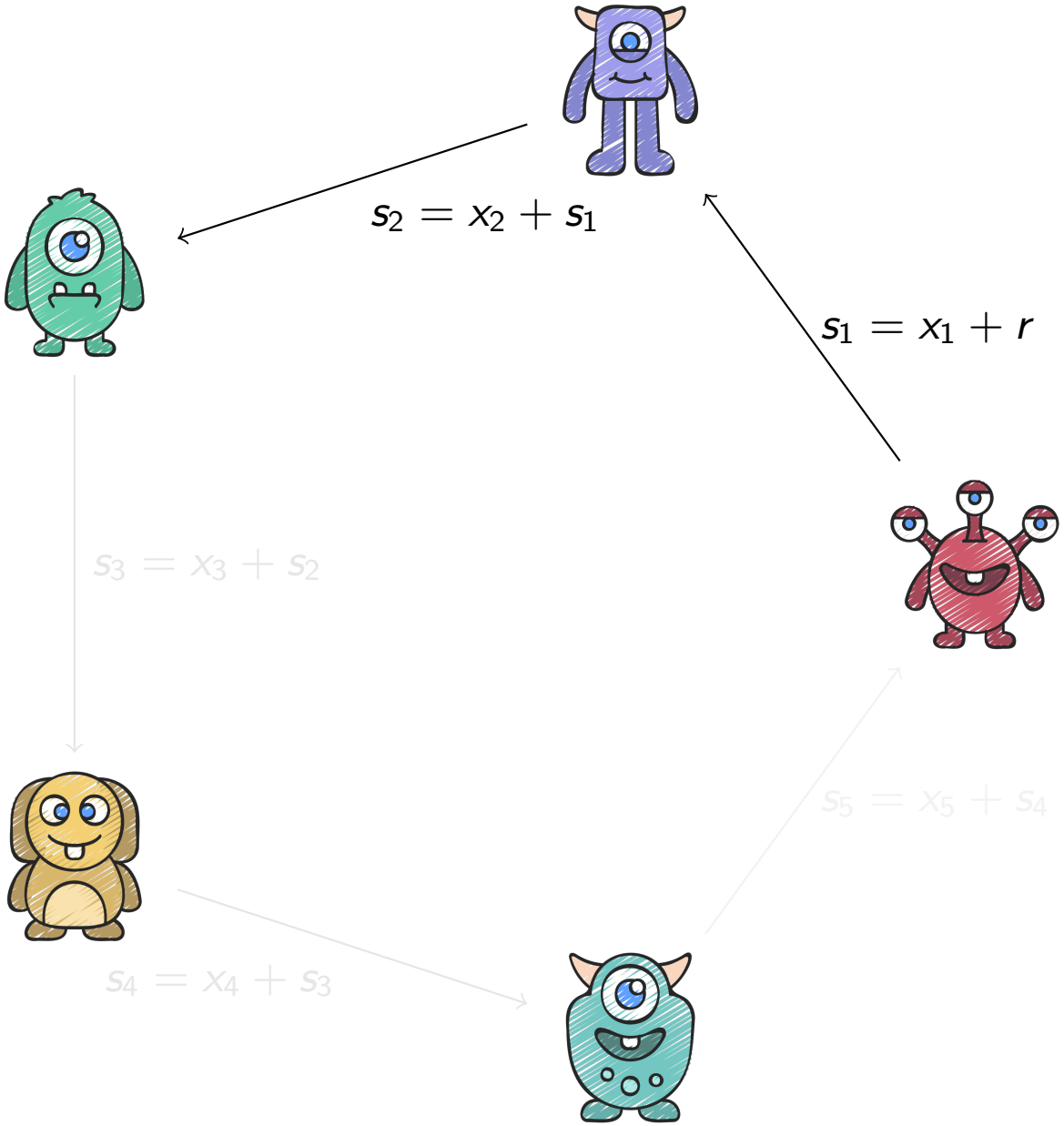
$$s_1 = x_1 + r \bmod q$$

  to P_2
- For $i \in \{2, \ldots, n-1\}$, P_i sends $s_i = s_{i-1} + x_i \bmod q$ to P_{i+1}
- P_n sends $s_n = s_{n-1} + x_n \bmod q$ to P_1
- P_1 computes $s = s_n - r \bmod q$ and **broadcasts** s/n to all parties

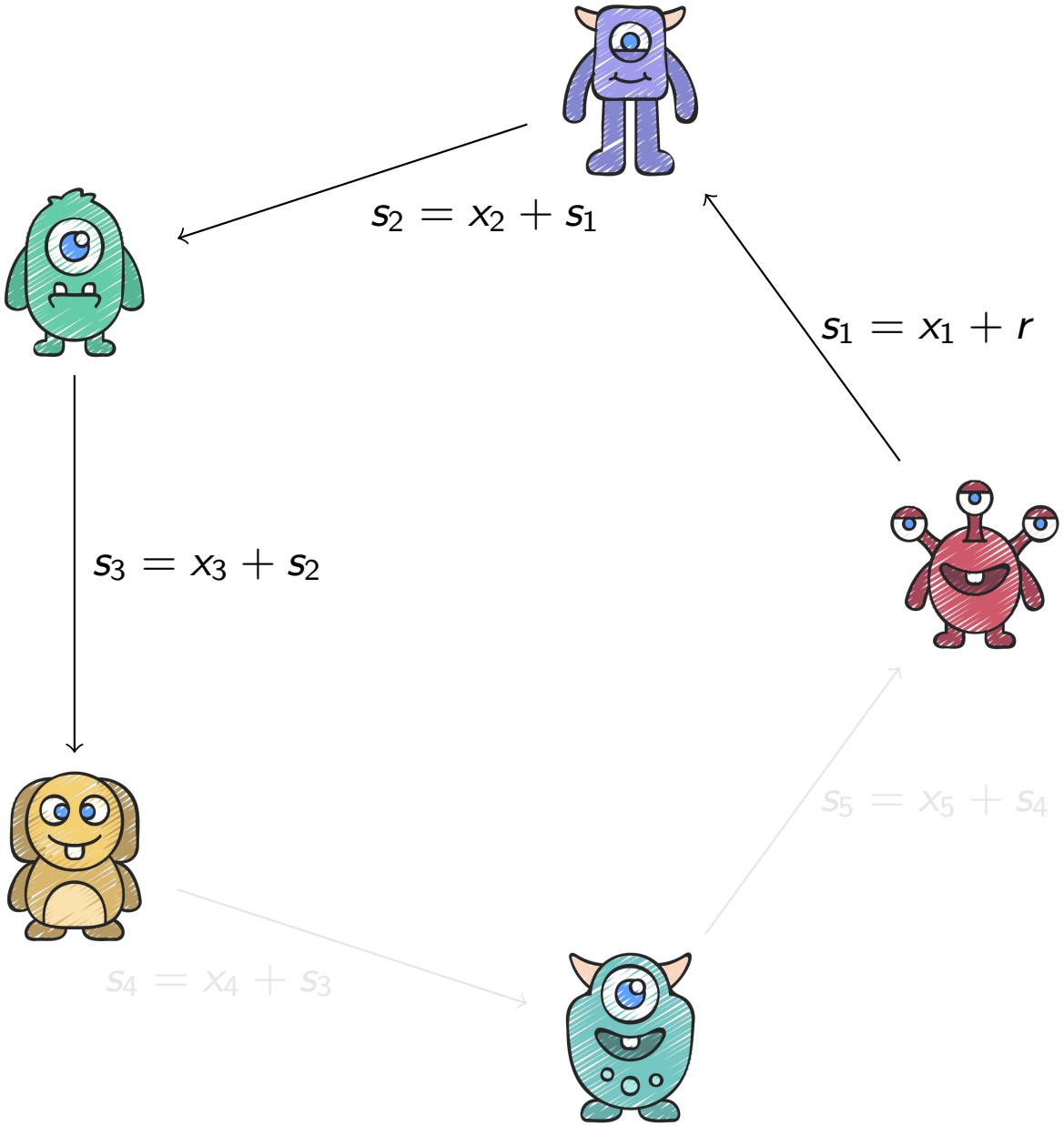
Mean computation



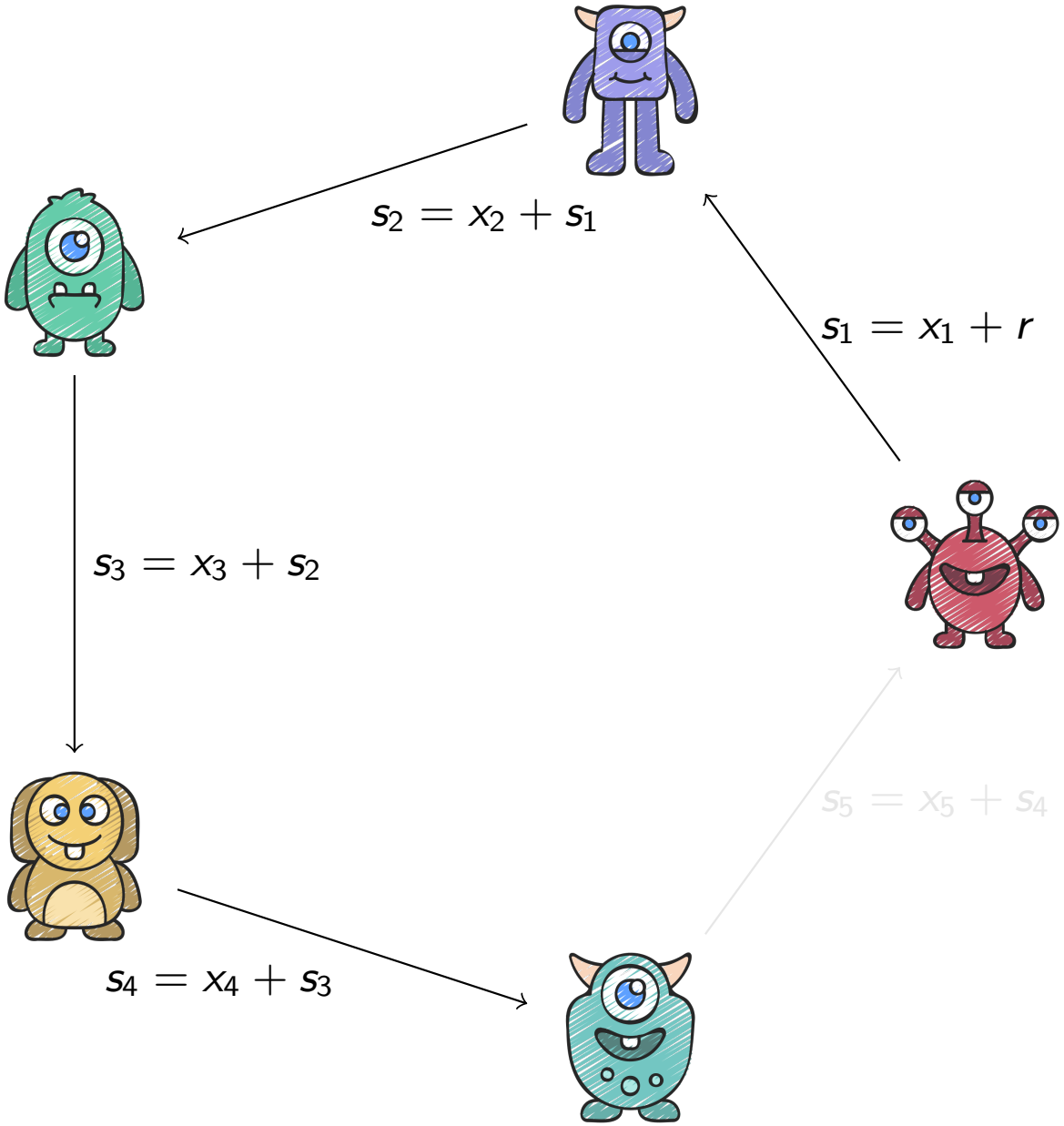
Mean computation



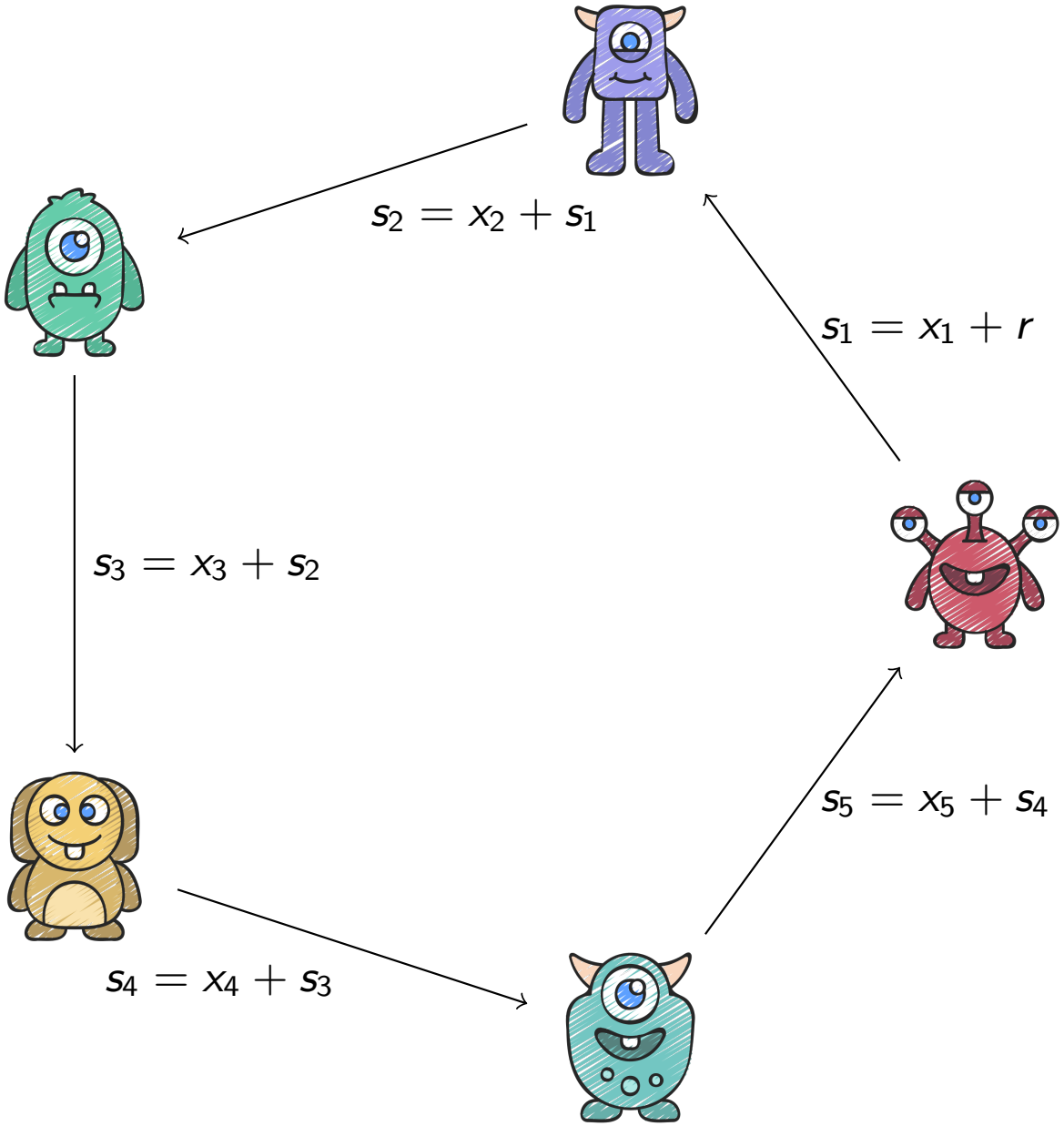
Mean computation



Mean computation



Mean computation



Mean computation

- perfect security in the presence of a semi-honest adversary controlling $t \leq 1$ parties

  $\rightsquigarrow$ **one-time pad** !

- no security in the presence of a semi-honest adversary controlling $t \geq 2$ parties

  $\rightsquigarrow$ **corrupt** P_2 **and** P_4**:** $x_3 = s_3 - s_2$!

- no security in the presence of a malicious adversary controlling $t \geq 1$ parties

  $\rightsquigarrow$ **send a (single) wrong value** s_i

Mean computation

- perfect security in the presence of a semi-honest adversary controlling $t \leq 1$ parties

  $\rightsquigarrow$ **one-time pad** !

- no security in the presence of a semi-honest adversary controlling $t \geq 2$ parties

  $\rightsquigarrow$ **corrupt** P_2 **and** P_4**:** $x_3 = s_3 - s_2$!

- no security in the presence of a malicious adversary controlling $t \geq 1$ parties

  $\rightsquigarrow$ **send a (single) wrong value** s_i

Mean computation

- perfect security in the presence of a semi-honest adversary controlling $t \leq 1$ parties

  $\rightsquigarrow$ **one-time pad** !

- no security in the presence of a semi-honest adversary controlling $t \geq 2$ parties

  $\rightsquigarrow$ **corrupt** P_2 **and** P_4**:** $x_3 = s_3 - s_2$!

- no security in the presence of a malicious adversary controlling $t \geq 1$ parties

  $\rightsquigarrow$ **send a (single) wrong value** s_i

Mean computation

- perfect security in the presence of a semi-honest adversary controlling $t \leq 1$ parties

  $\rightsquigarrow$ **one-time pad** !

- no security in the presence of a semi-honest adversary controlling $t \geq 2$ parties

  $\rightsquigarrow$ **corrupt** P_2 **and** P_4**:** $x_3 = s_3 - s_2$!

- no security in the presence of a malicious adversary controlling $t \geq 1$ parties

  $\rightsquigarrow$ **send a (single) wrong value** s_i

Mean computation

- perfect security in the presence of a semi-honest adversary controlling $t \leq 1$ parties

  $\rightsquigarrow$ **one-time pad** !

- no security in the presence of a semi-honest adversary controlling $t \geq 2$ parties

  $\rightsquigarrow$ **corrupt** P_2 **and** P_4**:** $x_3 = s_3 - s_2$!

- no security in the presence of a malicious adversary controlling $t \geq 1$ parties

  $\rightsquigarrow$ **send a (single) wrong value** s_i

Mean computation

- perfect security in the presence of a semi-honest adversary controlling $t \leq 1$ parties

  $\rightsquigarrow$ **one-time pad** !

- no security in the presence of a semi-honest adversary controlling $t \geq 2$ parties

  $\rightsquigarrow$ **corrupt** P_2 **and** P_4**:** $x_3 = s_3 - s_2$!

- no security in the presence of a malicious adversary controlling $t \geq 1$ parties

  $\rightsquigarrow$ **send a (single) wrong value** s_i

Mean computation in $O(1)$ rounds

- Each P_i shares its input x_i using a n-out-of-n **secret sharing**:

$$x_i = x_i^{(1)} + x_i^{(2)} + \cdots + x_i^{(n)} \bmod q$$

  and sends x_i^j to P_j (for $j \in \{1, \ldots, n\}$).

- Each P_i computes

$$s_i = x_1^{(i)} + x_2^{(i)} + \cdots + x_n^{(i)} \bmod q$$

  and **broadcasts** this value to all parties

- Each party computes

$$s = s_1 + s_2 + \cdots + s_n \bmod q$$

  and returns s/n as the **mean** of the inputs.

Mean computation in $O(1)$ rounds

- Each P_i shares its input x_i using a n-out-of-n **secret sharing**:

$$x_i = x_i^{(1)} + x_i^{(2)} + \cdots + x_i^{(n)} \bmod q$$

  and sends x_i^j to P_j (for $j \in \{1, \ldots, n\}$). Input sharing

- Each P_i computes

$$s_i = x_1^{(i)} + x_2^{(i)} + \cdots + x_n^{(i)} \bmod q$$

  and **broadcasts** this value to all parties

- Each party computes

$$s = s_1 + s_2 + \cdots + s_n \bmod q$$

  and returns s/n as the **mean** of the inputs.

Mean computation in $O(1)$ rounds

- Each P_i shares its input x_i using a n-out-of-n **secret sharing**:

$$x_i = x_i^{(1)} + x_i^{(2)} + \cdots + x_i^{(n)} \bmod q$$

and sends x_i^j to P_j (for $j \in \{1, \ldots, n\}$). Input sharing

- Each P_i computes

$$s_i = x_1^{(i)} + x_2^{(i)} + \cdots + x_n^{(i)} \bmod q$$

and **broadcasts** this value to all parties Computation

- Each party computes

$$s = s_1 + s_2 + \cdots + s_n \bmod q$$

and returns s/n as the **mean** of the inputs.

Mean computation in $O(1)$ rounds

- Each P_i shares its input x_i using a n-out-of-n **secret sharing**:

$$x_i = x_i^{(1)} + x_i^{(2)} + \cdots + x_i^{(n)} \bmod q$$

and sends x_i^j to P_j (for $j \in \{1, \ldots, n\}$). Input sharing

- Each P_i computes

$$s_i = x_1^{(i)} + x_2^{(i)} + \cdots + x_n^{(i)} \bmod q$$

and **broadcasts** this value to all parties Computation

- Each party computes

$$s = s_1 + s_2 + \cdots + s_n \bmod q$$

and returns s/n as the **mean** of the inputs. Reconstruction

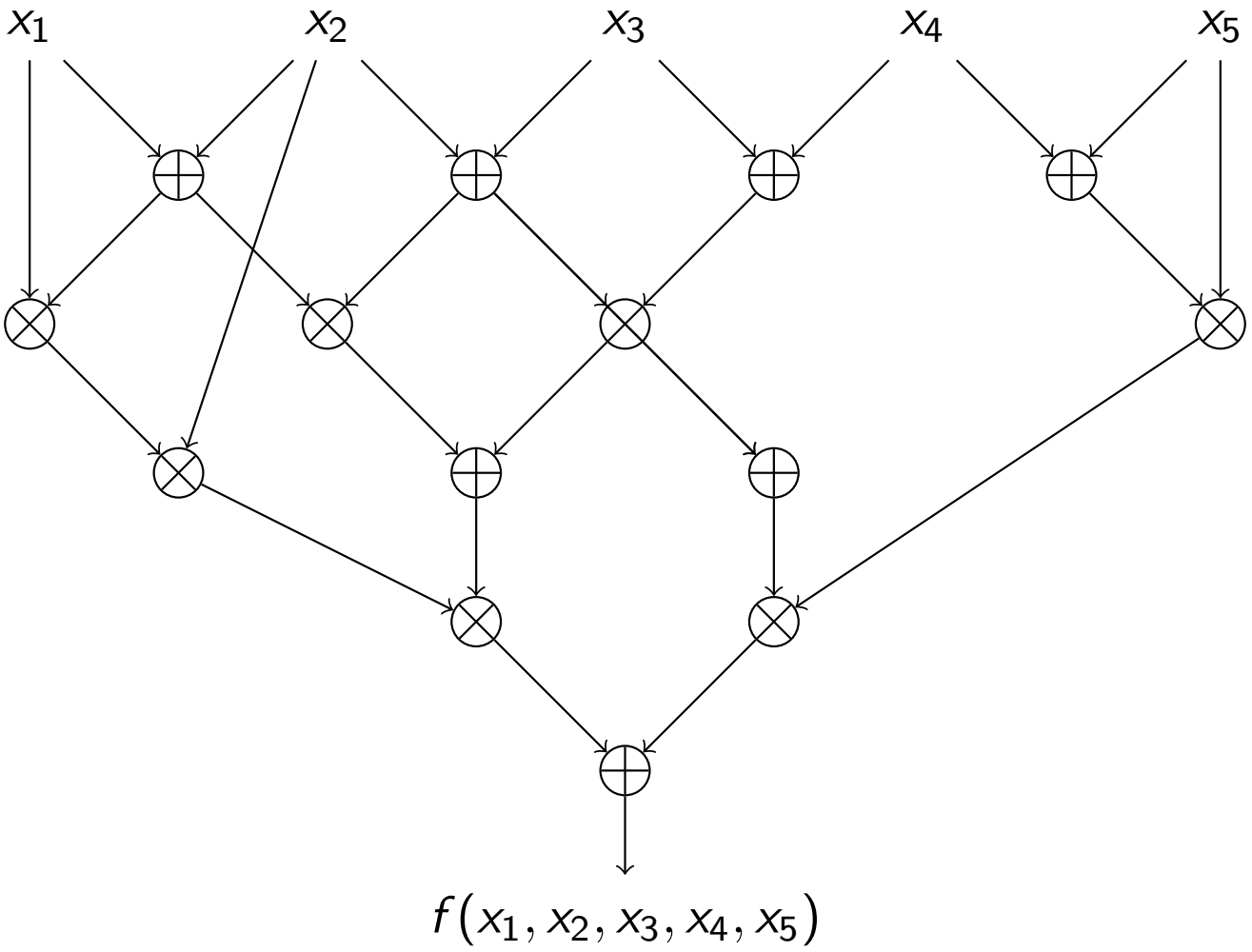
Ben-Or-Goldwasser-Wigderson Protocol (1988)

Semi-Honest Adversaries

For every n-ary function $f(x_1, \ldots, x_n)$, there exists a protocol for computing with perfect security in the presence of a semi-honest adversary controlling $t < n/2$ parties

- players represent f using an arithmetic circuit over a field $\mathbb{F}$
- $\#\mathbb{F} > n$
- each wire of the circuit gets a value in $\mathbb{F}$
- Gates:
  - **addition gate**: $g(a, b) = a + b$
  - **multiplication by a constant gate**: $g_c(a) = c \cdot a$
  - **multiplication gate**: $g(a, b) = a \cdot b$

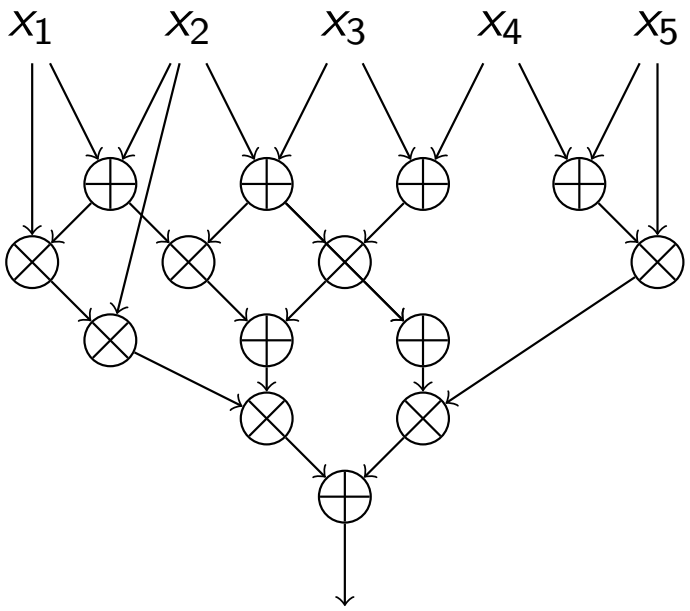
BGW Protocol - Circuit Evaluation



BGW Protocol - Circuit Evaluation

- In the secure protocol, each input wire is known to **only one** party
- And that party wants to keep it private!
- Moreover, we cannot reveal any intermediate values
- All values on all wires during the evaluation should be hidden
- Only values on the **output** wires should be revealed

BGW Protocol - Key Idea

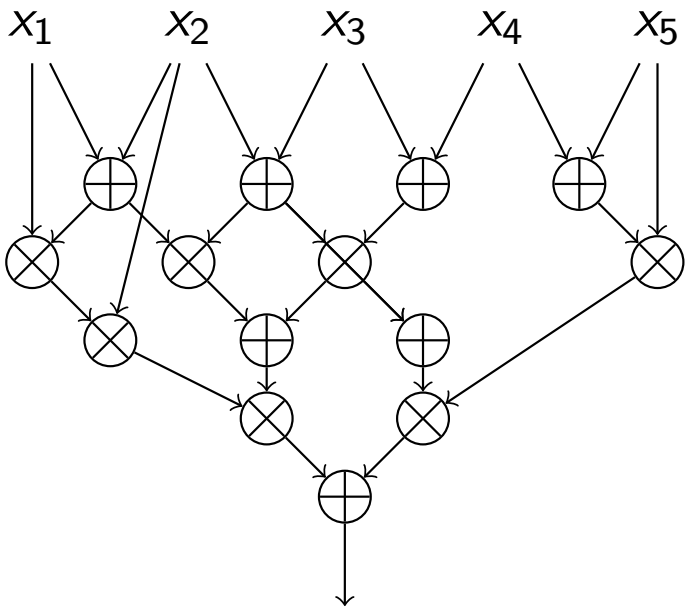


$f(x_1, x_2, x_3, x_4, x_5)$

- The parties will emulate a computation of the circuit on the inputs
- **Invariant:** The value of each wire is secret-shared among the parties

  $\rightsquigarrow$ **Shamir's secret sharing** with a (random) polynomial of degree t (with $t < n/2$)

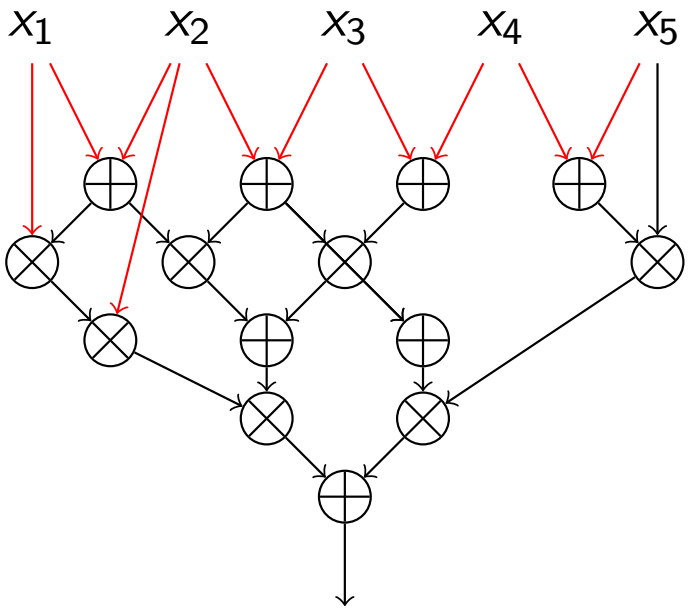
BGW Protocol - Overview



$f(x_1, x_2, x_3, x_4, x_5)$

- **Stage I:**
  $\rightsquigarrow$ Input sharing phase
- **Stage II:**
  $\rightsquigarrow$ Circuit emulation phase
- **Stage III:**
  $\rightsquigarrow$ Output reconstruction phase

BGW - Stage I - Input sharing phase



$f(x_1, x_2, x_3, x_4, x_5)$

- Each party P_i shares its input x_i
  - It chooses a random polynomial Q_i of degree t for which

$$Q_i(0) = x_i$$

  - It sends to each P_j the share $Q_i(j)$
- At the end of this stage each party P_i holds **shares**

$$Q_1(i), Q_2(i), \ldots, Q_n(i)$$

BGW - Stage II - Circuit Emulation

How to compute addition gates?

- **Input:** $a, b \in \mathbb{F}$
- **Output:** $g(a, b) = a + b \in \mathbb{F}$

- **Invariant**: There exists polynomials Q_a and Q_b s.t.:
  - $\deg(Q_a) \leq t$, $\deg(Q_b) \leq t$
  - $Q_a(0) = a$ and $Q_b(0) = b$
  - Each P_i knows $Q_a(i)$ and $Q_b(i)$

- **Computation:** for $Q_{a+b}(X) := Q_a(X) + Q_b(X)$, we have:
  - $\deg(Q_{a+b}) \leq t$
  - $Q_{a+b}(0) = Q_a(0) + Q_b(0) = a + b$
  - Each P_i can compute (locally) $Q_{a+b}(i) = Q_a(i) + Q_b(i)$

BGW - Stage II - Circuit Emulation

How to compute multiplication by constant gates?

- **Input:** $a, b \in \mathbb{F}$
- **Output:** $g_c(a) = a \cdot c \in \mathbb{F}$

- **Invariant**: There exists a polynomial Q_a s.t.:
  - $\deg(Q_a) \leq t$
  - $Q_a(0) = a$
  - Each P_i knows $Q_a(i)$

- **Computation:** for $Q_{ac}(X) := c \cdot Q_a(X)$, we have:
  - $\deg(Q_{ac}) \leq t$
  - $Q_{ac}(0) = c \cdot Q_a(0) = a \cdot c$
  - Each P_i can compute (locally) $Q_{ac}(i) = c \cdot Q_a(i)$

BGW - Stage II - Circuit Emulation

How to compute multiplication gates?

- **Input:** $a, b \in \mathbb{F}$
- **Output:** $g(a, b) = a \cdot b \in \mathbb{F}$

- **Invariant**: There exists polynomials Q_a and Q_b s.t.:
  - $\deg(Q_a) \leq t$, $\deg(Q_b) \leq t$
  - $Q_a(0) = a$ and $Q_b(0) = b$
  - Each P_i knows $Q_a(i)$ and $Q_b(i)$

- **Computation:** for $Q_{a \cdot b}(X) := Q_a(X)Q_b(X)$, we have:
  - Each P_i can compute (locally) $Q_{a \cdot b}(i) = Q_a(i) \cdot Q_b(i)$
  - but, $\deg(Q_{ab})$ can be **larger** than t!
  - but, $Q_{a \cdot b}$ is not a **random** polynomial!

BGW - Multiplication Gates

- **Reminder:** By Lagrange interpolation, for any polynomial P of degree $d < n$, there exists constants $\lambda_1, \ldots, \lambda_n$ s.t.

$$P(0) = \lambda_1 P(1) + \cdots + \lambda_n P(n)$$

$$\begin{pmatrix} 1 & 1 & 1 & 1 & \ldots & 1 \\ 1 & 2 & 4 & 8 & \ldots & 2^d \\ 1 & 3 & 9 & 27 & \ldots & 3^d \\ \vdots & & & & & \vdots \\ 1 & n & n^2 & n^3 & \ldots & n^d \end{pmatrix} \cdot \begin{pmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_d \end{pmatrix} = \begin{pmatrix} P(1) \\ P(2) \\ P(3) \\ \vdots \\ P(n) \end{pmatrix}$$

BGW - Multiplication Gates

- **Reminder:** By Lagrange interpolation, for any polynomial P of degree $d < n$, there exists constants $\lambda_1, \ldots, \lambda_n$ s.t.

$$P(0) = \lambda_1 P(1) + \cdots + \lambda_n P(n)$$

$$\begin{pmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_d \end{pmatrix} = \begin{pmatrix} \lambda_1 & \lambda_2 & \lambda_3 & \ldots & \ldots & \lambda_n \\ \ldots & \ldots & \ldots & \ldots & \ldots & \ldots \\ \vdots & & & & & \vdots \\ \ldots & \ldots & \ldots & \ldots & \ldots & \ldots \end{pmatrix} \cdot \begin{pmatrix} P(1) \\ P(2) \\ P(3) \\ \vdots \\ P(n) \end{pmatrix}$$

BGW - Multiplication Gates

- **Reminder:** By Lagrange interpolation, for any polynomial P of degree $d < n$, there exists constants $\lambda_1, \ldots, \lambda_n$ s.t.

$$P(0) = \lambda_1 P(1) + \cdots + \lambda_n P(n)$$

$$\begin{pmatrix} P(0) \\ p_1 \\ p_2 \\ \vdots \\ p_d \end{pmatrix} = \begin{pmatrix} \lambda_1 & \lambda_2 & \lambda_3 & \ldots & \ldots & \lambda_n \\ \ldots & \ldots & \ldots & \ldots & \ldots & \ldots \\ \vdots & & & & & \vdots \\ \ldots & \ldots & \ldots & \ldots & \ldots & \ldots \end{pmatrix} \cdot \begin{pmatrix} P(1) \\ P(2) \\ P(3) \\ \vdots \\ P(n) \end{pmatrix}$$

BGW - Multiplication Gates

- For $Q_{a \cdot b}(X) := Q_a(X)Q_b(X)$, $\deg(Q_{ab})$ is at most $2t < n$ so there exist (public) constants $\lambda_1, \ldots, \lambda_n$ s.t.

$$Q_{ab}(0) = \lambda_1 Q_{ab}(1) + \cdots + \lambda_n Q_{ab}(n)$$

- Each P_i can compute (locally) $Q_{a \cdot b}(i) = Q_a(i) \cdot Q_b(i)$

- Each party P_i shares the value $Q_{a \cdot b}(i)$
  - It chooses a random polynomial H_i of degree t for which

$$H_i(0) = Q_{a \cdot b}(i)$$

  - It sends to each P_j the share $H_i(j)$
- At the end of this stage each party P_i holds **shares**

$$H_1(i), H_2(i), \ldots, H_n(i)$$

and computes as its share of $a \cdot b$

$$\lambda_1 H_1(i) + \cdots + \lambda_n H_n(i)$$

BGW - Multiplication Gates

Why Does It Work?

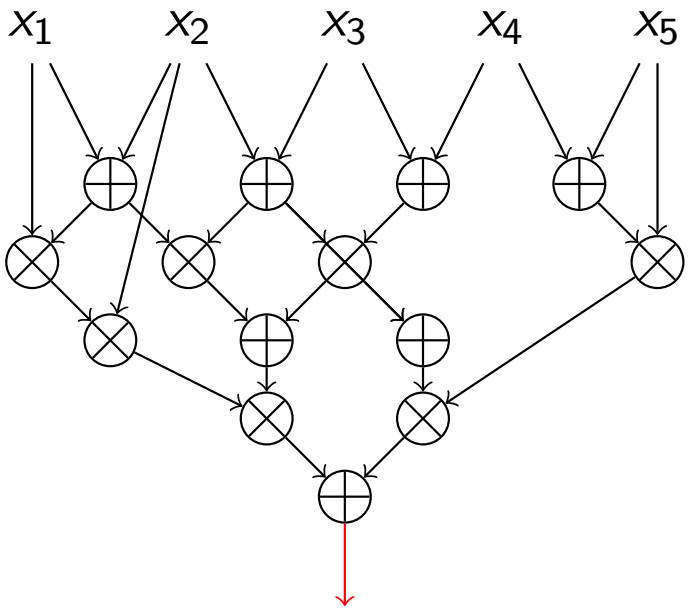
- The parties obtains the shares for the polynomial

$$H(X) = \lambda_1 H_1(X) + \cdots + \lambda_n H_n(X)$$

- This is by construction a polynomial of degree at most t
- It is random (since each of the H_i's are random)
- Its constant term is

$$\begin{aligned} H(0) &= \lambda_1 H_1(0) + \cdots + \lambda_n H_n(0) \\ &= \lambda_1 Q_{ab}(1) + \cdots + \lambda_n Q_{ab}(n) \\ &= Q_{ab}(0) = ab \end{aligned}$$

BGW - Stage III - Output Reconstruction Phase



$f(x_1, x_2, x_3, x_4, x_5)$

- Each party P_i hold a share of the output $f(x_i, \ldots, x_n)$
- Each party broadcasts its share to all other parties and they can reconstruct $f(x_i, \ldots, x_n)$

Ben-Or-Goldwasser-Wigderson Protocol (1988)

Semi-Honest Adversaries

For every n-ary function $f(x_1, \ldots, x_n)$, there exists a protocol for computing with perfect security in the presence of a semi-honest adversary controlling $t < n/2$ parties

- why do we need $t < n/2$?
- **Extension:** use of "Preprocessed Multiplication Triples"
- **Application:** construction of Zero-Knowledge protocols

Ben-Or-Goldwasser-Wigderson Protocol (1988)

Semi-Honest Adversaries

For every n-ary function $f(x_1, \ldots, x_n)$, there exists a protocol for computing with perfect security in the presence of a semi-honest adversary controlling $t < n/2$ parties

- why do we need $t < n/2$?
- **Extension:** use of "Preprocessed Multiplication Triples"
- **Application:** construction of Zero-Knowledge protocols

Ben-Or-Goldwasser-Wigderson Protocol (1988)

Semi-Honest Adversaries

For every n-ary function $f(x_1, \ldots, x_n)$, there exists a protocol for computing with perfect security in the presence of a semi-honest adversary controlling $t < n/2$ parties

- why do we need $t < n/2$?
- **Extension:** use of "Preprocessed Multiplication Triples"
- **Application:** construction of Zero-Knowledge protocols